

SERVICIOS TELEMÁTICOS

Memoria Práctica Final

Profesor: Luis Bernal Escobedo

Autores:

- **Rafael Guillén García.**
 - **DNI:** [REDACTED]
 - **Grupo:** 3.3.
 - **Correo:** rafael.guilleng@um.es
- **Daniel Fernández Ayala.**
 - **DNI:** [REDACTED]
 - **Grupo:** 3.3.
 - **Correo:** daniel.f.a@um.es

Fecha de Entrega: 10 de mayo de 2025.

ÍNDICE

1.	Introducción.....	3
2.	Descripción del escenario	4
2.1.	Versiones de software	5
3.	Programación Web-SSTT HTTP Server.....	5
3.1.	Recepción de parámetros y creación del socket	5
3.2.	Bucle principal y atención de conexiones	6
3.3.	Persistencia HTTP por tiempo	6
3.4.	Análisis de la petición HTTP	7
3.5.	Localización y envío de recursos.....	8
3.6.	Construcción de la respuesta HTTP	8
3.7.	Gestión del formulario GET	8
3.8.	Gestión de cookies.....	9
3.9.	Tratamiento de errores HTTP	10
4.	Despliegue del servicio DNS.....	10
4.1.	Preparación de la resolución local en servidor y en cliente.....	10
4.2.	Instalación y configuración del servidor DNS primario.....	11
4.3.	Creación y contenido del fichero de zona	12
4.4.	Configuración del servidor DNS secundario.....	13
5.	Despliegue del servicio SMTP/POP	13
5.1.	Preparación de la resolución DNS para el servicio de correo	13
5.2.	Instalación y configuración del servicio SMTP con Exim4	14
5.3.	Instalación y configuración del servicio POP3 con Dovecot	14
5.4.	Creación de usuarios y buzones Maildir	15
5.5.	Configuración de Thunderbird	15
6.	Despliegue del servicio HTTP/HTTPS con Apache.....	16
6.1.	Configuración realizada: HTTP.....	16
6.2.	Configuración realizada: HTTPS	18
6.2.1.	Preparación de la PKI y creación de la CA	18
6.2.2.	Generación del certificado del servidor.....	19
6.2.3.	Configuración HTTPS de Apache	20
6.2.4.	Configuración del navegador cliente para confiar en la CA	21

6.3.	Mejora adicional: Autenticación de cliente mediante certificado	21
6.3.1.	Generación de las claves privadas y de las CSRs.....	21
6.3.2.	Firma de los certificados por parte de la CA	22
6.3.3.	Empaquetado de los certificados en formato PKCS#12.....	22
6.3.4.	Configuración de Apache para exigir los certificados.....	23
6.3.5.	Importación de los certificados en Firefox	23
6.3.6.	Verificación del acceso con autenticación de cliente.....	24
7.	Despliegue del servicio IPsec	25
7.1.	Instalación de strongSwan	25
7.2.	Generación del certificado nuevo para el extremo cliente	25
7.3.	Copia de certificados a los directorios de strongSwan	26
7.4.	Configuración de /etc/ipsec.conf en el servidor	26
7.5.	Configuración de /etc/ipsec.conf en el cliente	27
7.6.	Configuración de /etc/ipsec.secrets	28
7.7.	Activación del servicio y comprobación	29
8.	Trazas del escenario.....	29
8.1.	Traza del servicio HTTP desarrollado en Python	29
8.1.1.	Traza del formulario GET del servidor.....	30
8.1.2.	Traza de cookies y bloqueo de acceso	30
8.1.3.	Traza de solicitud de recurso inexistente.....	31
8.2.	Traza DNS y acceso a http://www.fontanerosvillanueva6493.org	31
8.3.	Traza DNS y acceso a https://www.fontanerosvillanueva6493.org	32
8.4.	Traza DNS, SMTP y POP	33
8.4.1.	DNS y SMTP.....	33
8.4.2.	DNS y POP	33
8.4.3.	Consulta al registro MX	34
8.5.	IKE y IPsec	34
9.	Problemas encontrados	36
10.	Horas empleadas (aproximadas).....	37
11.	Conclusión y valoración personal	37

1. Introducción

En esta práctica se desarrolla el despliegue y la configuración de varios servicios telemáticos integrados en una organización, trabajando en un escenario formado por una máquina servidor y una máquina cliente. El objetivo principal de la práctica es construir una infraestructura funcional y coherente que permita ofrecer distintos servicios de red bajo un mismo dominio, de forma que todos estos servicios queden relacionados entre sí y puedan utilizarse dentro del escenario planteado. Esta organización se ha definido con el dominio **fontanerosvillanueva6493.org**.

El documento se centra en la descripción de las configuraciones realizadas y en la explicación de los servicios desplegados.

Primero, se ha desarrollado un servicio web HTTP en Python, incluyendo la gestión de peticiones, persistencia, cookies y tratamiento de errores.

Después, se ha desplegado el servicio DNS de la organización, encargado de resolver los nombres de los equipos y de los servicios definidos en el escenario. Para ello, se ha configurado un servidor DNS primario en la máquina servidor y uno secundario en la máquina cliente, de forma que ambos permitan resolver los nombres del dominio y mantener el servicio disponible en caso de que el servidor primario se caiga.

Tras esto, se describe la configuración del servicio de correo electrónico mediante SMTP y POP3 y su integración con DNS y con las cuentas de usuario del sistema. Después, se explica el despliegue del servidor Apache para acceso HTTP y HTTPS, junto con la infraestructura de certificados X.509 utilizada para la autenticación del servidor y, como mejora adicional, para la autenticación de cliente.

Por último, se describe la configuración de IPsec entre cliente y servidor, utilizando IKEv2, certificados X.509 y una asociación de seguridad en modo transporte mediante la cabecera AH.

Por tanto, este documento recoge todo el desarrollo de la configuración completa del escenario. Además, tras las explicaciones de las configuraciones, se incluyen trazas de Wireshark de los protocolos utilizados, los problemas encontrados, las horas aproximadas empleadas y una valoración personal sobre la práctica.

2. Descripción del escenario

El escenario de la práctica se ha desplegado sobre dos máquinas virtuales conectadas en una misma red interna. Sobre esta topología se ha definido una organización que tiene dominio propio.

La organización se identifica con el dominio **fontanerosvillanueva6493.org**, de forma que todos los servicios desplegados utilizan este dominio tanto en nombres de host como en las direcciones de correo de los usuarios.

La máquina servidor tiene asignada la dirección IP **192.168.56.102** y en ella se encuentran los servicios principales de la organización como el servidor Web-SSTT HTTP desarrollado en Python, se aloja el servidor DNS primario, se despliega el servicio de correo electrónico mediante Exim4 y Dovecot, se configura el servidor Apache para acceso HTTP y HTTPS y se mantiene la CA utilizada para generar los certificados X.509 del escenario. Además, en esta máquina se ha configurado uno de los extremos de la protección IPsec establecida entre cliente y servidor.

La máquina cliente tiene asignada la dirección IP **192.168.56.101** y actúa como equipo desde el que se accede a los servicios desplegados. En ella se utilizan el navegador web, Thunderbird y Wireshark para realizar las pruebas del escenario. Además, esta máquina actúa también como servidor DNS secundario del dominio y constituye el otro extremo de la asociación de seguridad IPsec configurada con el servidor.

Dentro del dominio se han definido los nombres necesarios para cada equipo y para los distintos servicios desplegados.

- La máquina servidor se identifica como: **servidor.fontanerosvillanueva6493.org**
- La máquina cliente se identifica como: **cliente.fontanerosvillanueva6493.org**

A partir de estos nombres se han definido también los alias:

- **www.fontanerosvillanueva6493.org**
- **web.fontanerosvillanueva6493.org**
- **smtp.fontanerosvillanueva6493.org**
- **pop.fontanerosvillanueva6493.org**
- **dns1.fontanerosvillanueva6493.org**
- **dns2.fontanerosvillanueva6493.org**

En cuanto a los usuarios, se han creado dos cuentas de correo pertenecientes al dominio de la organización:

- **rafael_64@fontanerosvillanueva6493.org**
- **daniel_93@fontanerosvillanueva6493.org**

Estas cuentas se han definido como usuarios del sistema en la máquina servidor, de forma que sus directorios personales contienen los buzones en formato Maildir utilizados por los servicios SMTP y POP3.

2.1. Versiones de software

- Sistema operativo máquina servidor: Ubuntu 16.04.7 LTS
- Sistema operativo máquina cliente: Ubuntu 16.04.1 LTS
- Python: 3.5.2
- Apache2: 2.4.18 (Ubuntu)
- BIND9: 9.10.3-P4-Ubuntu
- Exim4: 4.86_2
- Dovecot: 2.2.22
- OpenSSL: 1.0.2g
- Firefox:45.0.2
- Thunderbird: 68.10.0
- strongSwan: U5.3.5/K4.4.0-210-generic

3. Programación Web-SSTT HTTP Server

En este apartado se describe la implementación de un servidor HTTP en Python con el que se puede atender una petición web desde el navegador, devolver una página principal y gestionar varios de los elementos más importantes, como el acceso a recursos, formulario GET, gestión de cookies, persistencia de la conexión y tratamiento de errores.

Primero, se ha preparado el contenido web mínimo que sirve el servidor. Para ello, se tiene un fichero **index.html** que actúa como página principal y en el que se incluye un encabezado, una imagen y un formulario para introducir un correo electrónico. De esta forma, al acceder al recurso principal se devuelve esta página sencilla.

```
<html>
<head>
  <title>Servidor Simple para Servicios Telemáticos</title>
</head>
<body bgcolor="lightgray">
  <h1>Esta es una pagina de prueba</h1>

  
  <p>Es un ejemplo simple pero funcional</p>

  <form method="GET">
    Correo electrónico:<br>
    <input type="text" name="correo" value=""><br>
    <input type="submit" value="Enviar">
  </form>
</body>
</html>
```

Figura 1. Contenido del fichero index.html.

3.1. Recepción de parámetros y creación del socket

Ahora, en cuanto a la implementación del servidor, se ha utilizado un socket TCP en Python. Para ello, se crea el socket activando la opción **SO_REUSEADDR**, se hace el **bind** con la dirección y el puerto indicados, y finalmente, el servidor queda a la escucha de conexiones entrantes. Cuando un cliente establece conexión, el servidor la acepta y pasa a gestionar la comunicación con ese cliente.

Los parámetros del programa se reciben mediante **argparse**. En estos parámetros, se indican la dirección IP, el puerto y el directorio base desde el que se servirán los recursos. En el caso actual, se utilizará la IP **192.168.56.102** (máquina servidor) y el puerto **9000**.

```
# Argument parser para obtener la ip y puerto de los parámetros de ejecución del programa. IP por defecto 0.0.0.0
parser = argparse.ArgumentParser()
parser.add_argument("-p", "--port", help="Puerto del servidor", type=int, required=True)
parser.add_argument("-ip", "--host", help="Dirección IP del servidor o localhost", required=True)
parser.add_argument("-wb", "--webroot", help="Directorio base desde donde se sirven los ficheros (p.ej. /home/user/mi_web)")
parser.add_argument("--verbose", "-v", action='store_true', help='Incluir mensajes de depuración en la salida')
args = parser.parse_args()

if args.verbose:
    logger.setLevel(logging.DEBUG)

logger.info('Enabling server in address {} and port {}'.format(args.host, args.port))

logger.info("Serving files from {}".format(args.webroot))

# * Crea un socket TCP (SOCK_STREAM)
server = socket.socket(family=socket.AF_INET, type=socket.SOCK_STREAM, proto=0)
# * Permite reusar la misma dirección previamente vinculada a otro proceso. Debe ir antes de sock.bind
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
# * Vinculamos el socket a una IP y puerto elegidos
server.bind((args.host, args.port))
# * Escucha conexiones entrantes
server.listen()
```

Figura 2. Recepción de parámetros y creación del socket TCP del servidor.

3.2. Bucle principal y atención de conexiones

Más específicamente, tras lanzar el servidor, este entra en un bucle principal en el que permanece esperando nuevas conexiones. Cuando llega una conexión desde el cliente, se acepta y se crea un proceso hijo mediante **fork**, de forma que ese proceso hijo es el que se encarga de gestionar la petición correspondiente, mientras que el proceso padre continúa escuchando nuevas conexiones. Por esto, el servidor puede atender varias conexiones de manera independiente y el proceso principal no se queda bloqueado mientras se procesa una petición.

```
## Bucle infinito para mantener el servidor activo indefinidamente
while True:
    # - Aceptamos la conexión
    conn, addr = server.accept()
    # - Creamos un proceso hijo
    pid = os.fork()
    # - Si es el proceso hijo se cierra el socket del padre y procesar la petición con process_web_request()
    if pid == 0:
        cerrar_conexion(server)
        process_web_request(conn, args.webroot)
        sys.exit(0)
    # - Si es el proceso padre cerrar el socket que gestiona el hijo.
    else:
        cerrar_conexion(conn)
```

Figura 3. Bucle principal del servidor y creación de procesos hijo para atender conexiones.

3.3. Persistencia HTTP por tiempo

Una vez aceptada la conexión, el tratamiento principal de las peticiones se realiza dentro de la función **process_web_request**. En esta parte, se utiliza **select.select** junto con un valor de tiempo de espera para mantener la conexión abierta durante un tiempo determinado. Mientras no se supere ese tiempo sin recibir nuevas peticiones, la misma conexión se reutiliza. Si durante ese intervalo no se recibe ningún mensaje nuevo, la conexión se cierra.

En este caso, se ha fijado un tiempo de persistencia de **32 segundos** (correspondiente a la suma de los últimos dígitos de los DNI +10: **6+4+9+3+10**), y ese valor se devuelve también en la cabecera Keep-Alive de la respuesta.

```
# * Bucle para esperar hasta que lleguen datos en la red a través del socket cs con select()
while True:
    """ * Se comprueba si hay que cerrar la conexión por exceder TIMEOUT_CONNECTION segundos
    sin recibir ningún mensaje o hay datos. Se utiliza select.select """
    rsublist, wsublist, xsublist = select.select([cs], [], [], TIMEOUT_CONNECTION)
    if not rsublist: # Si no hay leibles significa que saltó el timeout
        break

    # * Si no es por timeout y hay datos en el socket cs:
    else:
        # * Leer los datos con recv.
        mensaje = recibir_mensaje(cs)
        if not mensaje:
            break # Para que el bucle se rompa cuando se cierra la conexión

        mensaje_acumulado += mensaje

        # Comprobamos si la petición HTTP está completa (tiene la línea en blanco)
        if "\r\n\r\n" not in mensaje_acumulado:
            continue # si no está completa, esperamos datos
```

Figura 4. Implementación de la persistencia.

3.4. Análisis de la petición HTTP

Para procesar correctamente cada mensaje HTTP, el servidor va acumulando en un buffer los datos recibidos hasta encontrar la secuencia `\r\n\r\n`, que marca el final de las cabeceras. A partir de ahí, separa la línea de petición del resto de cabeceras y analiza su contenido. Este paso es importante ya que permite validar si la petición recibida tiene una estructura correcta antes de continuar con el tratamiento del recurso solicitado.

En primer lugar, se comprueba la línea inicial de la petición. Para ello, se utiliza una expresión regular que permite extraer el método, la ruta solicitada y la versión del protocolo. Si la versión no es **HTTP/1.1**, el servidor devuelve el error **505 HTTP Version Not Supported**. Después, se comprueba el método recibido, que aunque el funcionamiento principal gira en torno a peticiones GET, también se admite POST. Si el método no es válido, el servidor responde con el error **405 Method Not Allowed**. A continuación, se analizan las cabeceras una a una y se almacenan en un diccionario. Entre ellas, una de las comprobaciones más importantes es la presencia de la cabecera Host, ya que si no aparece se devuelve el error **400 Bad Request**.

```
# Separamos la petición
partes_mensaje = mensaje_acumulado.split("\r\n\r\n", 1)
peticion_actual = partes_mensaje[0]

# Guardamos lo demás por si es de una siguiente petición
if len(partes_mensaje) > 1:
    mensaje_acumulado = partes_mensaje[1]
else:
    mensaje_acumulado = ""

lineas = peticion_actual.split('\r\n') # Dividir el string

# * Analizar que la línea de solicitud y comprobar está bien formateada según HTTP 1.1
m = re_lineal.fullmatch(lineas[0])
if m:
    # * Comprobar si la versión de HTTP es 1.1
    if m.group('version') == "HTTP/1.1":
        # * Comprobar si es un método GET o POST, si no devolver Error 405 "Method Not Allowed".
        metodo = m.group('metodo')
        if not re_getpost.fullmatch(m.group('metodo')):
            procesar_error(cs, 405)
            continue
```

Figura 5. Análisis de la petición HTTP y validaciones de cabeceras

```

# Diccionario con las cabeceras y atributos
cabeceras = {}
for linea in lineas[1:]:
    if linea == "":
        break # Hemos llegado al fin de cabeceras
    m_cabecera = re_cabecera.fullmatch(linea)
    if m_cabecera:
        nombre = m_cabecera.group('header')
        valor_cabecera = m_cabecera.group('valor')
        cabeceras[nombre] = valor_cabecera
        print(nombre + ": " + valor_cabecera) # Imprimir las cabeceras

# Verificar Host
if 'Host' not in cabeceras:
    procesar_error(cs, 400)
    break # cerrar la conexión

```

Figura 6: Validación de cabeceras

3.5. Localización y envío de recursos

Ahora, para servir los recursos web, el servidor distingue si la ruta solicitada por el cliente es /. En ese caso, dicha ruta se asocia al fichero *index.html*. Después, se construye la ruta absoluta del recurso combinando el directorio base *webroot* con el nombre solicitado, y se comprueba si el fichero existe. Si no existe, se devuelve el error **404 Not Found**. Si sí existe, se obtiene su tamaño, se identifica su extensión y se determina el tipo correspondiente mediante su diccionario de tipos válidos.

Por último, se construye la respuesta **200 OK** y se envía el contenido del fichero al cliente en bloques binarios de tamaño 8KB. De esta forma, el servidor puede devolver tanto la página HTML como la imagen de dicha página y otros recursos.

3.6. Construcción de la respuesta HTTP

Las respuestas correctas generadas por el servidor incluyen las cabeceras principales del servicio: *Server*, *Content-Type*, *Content-Length*, *Date*, *Connection*, *Keep-Alive* y *Set-Cookie* cuando corresponde. Estas cabeceras se añaden al mensaje de respuesta antes del cuerpo del recurso, de forma que el cliente recibe el contenido solicitado y la información necesaria sobre el tipo de recurso, su tamaño y el comportamiento de la conexión.

```

""" * Preparar respuesta con código 200. Construir una respuesta que incluya: la línea de respuesta y
las cabeceras Date, Server, Connection, Set-Cookie (para la cookie cookie_counter),
Content-Length y Content-Type. """
if extension in filetypes: # Solo trata las extensiones válidas
    fecha_actual = datetime.now()
    fecha_formateada = fecha_actual.strftime('%Y-%m-%d %H:%M:%S') # El formato del login
    tipo_extension = filetypes[extension] # Extensión según el diccionario filetypes
    respuesta = "HTTP/1.1 200 OK\r\n"
    respuesta += "Server: fontanerosvillanueva6493.org (Ubuntu)\r\n"
    respuesta += "Content-Type: " + tipo_extension + "; charset=utf-8\r\n"
    respuesta += "Content-Length: " + str(tamano) + "\r\n"
    respuesta += "Date: " + fecha_formateada + "\r\n"
    respuesta += "Connection: Keep-Alive\r\n"
    respuesta += "Keep-Alive: timeout=" + str(TIMEOUT_CONNECTION) + ", max=100\r\n"
    respuesta += "Set-Cookie: cookie_counter_6493=" + str(contador_cookies) + "; Max-Age=30 \r\n"
    respuesta += "\r\n"

```

Figura 7. Creación de la respuesta por el servidor.

3.7. Gestión del formulario GET

Por otra parte, el formulario incluido en *index.html* pide al usuario un correo electrónico y utiliza el método **GET**, de forma que el dato introducido aparece dentro de la propia

URL. En el servidor, una vez separada la ruta de la cadena de parámetros, se busca el parámetro **correo** y se extrae su valor. Después, se decodifica el símbolo “@” y se compara el resultado con las direcciones de correo: rafael.guilleng@um.es y daniel.f.a@um.es. Si el valor coincide con alguna de estas direcciones, se genera una página HTML indicando que el correo es correcto, si no coincide se devuelve otra página indicando que el correo es incorrecto. Ambas respuestas se envían con código **200 OK**.

```
# Comprobar formulario del correo
m_correo = er_correo.search(parametros)
if m_correo:
    correo = m_correo.group(1).replace('%40', '@') # para decodificar la @
    # Comprobar si es nuestro correo
    if correo == CORREO_RAFAEL or correo == CORREO_DANIEL:
        html_formulario = "<html><body><h1>El correo es correcto</h1></body></html>"
    else:
        html_formulario = "<html><body><h1>El correo es incorrecto</h1></body></html>"
```

Figura 8: Código de la comprobación del formulario del correo

3.8. Gestión de cookies

Además, también se ha implementado una gestión básica de cookies para controlar el número de accesos a los recursos. Para ello, el servidor analiza la cabecera **Cookie** y busca una cookie con el nombre **cookie_counter_6493**. Si el cliente todavía no dispone de esa cookie, se inicializa el contador con valor 1. Si ya existe y su valor está entre 1 y 9, el servidor incrementa el contador en 1. Cuando el valor alcanza el máximo permitido (10), se deniega el acceso al contenido y se devuelve el error **403 Forbidden**. Para mantener este valor entre peticiones, en la respuesta se añade la cabecera **Set-Cookie** con el contador actualizado y con **Max-Age=30**, de forma que la cookie expira a los 30 segundos.

```
def process_cookies(headers, cs):
    # 1. Se analizan las cabeceras en headers para buscar la cabecera Cookie
    er_cookie = re.compile(r'Cookie')
    er_cookie_counter = re.compile(r'(?P<nombre>cookie_counter_6493)=(?P<valor>\d+)')
    for header, valor_header in headers.items():
        # 2. Una vez encontrada una cabecera Cookie se comprueba si el valor es cookie_counter
        if er_cookie.fullmatch(header):
            m = er_cookie_counter.search(valor_header)
            if m:
                valor = int(m.group('valor'))
                # 4. Si se encuentra y tiene el valor MAX_ACCESOS se devuelve MAX_ACCESOS
                if valor >= MAX_ACCESOS:
                    return MAX_ACCESOS
                # 5. Si se encuentra y tiene un valor 1 <= x < MAX_ACCESOS se incrementa en 1 y se devuelve el valor
                elif 1 <= valor < MAX_ACCESOS:
                    valor += 1
                    return valor
            # 3. Si no se encuentra cookie_counter, se devuelve 1
    return 1
```

Figura 9: Código de la función `process_cookies`

3.9. Tratamiento de errores HTTP

Finalmente, se ha implementado una función para el tratamiento de los errores HTTP: *procesar_error*. Esta función recibe un código de error y construye una página HTML con el mensaje correspondiente e incorpora una imagen asociada al código de error. Los errores que se pueden contemplar son *400 Bad Request*, *403 Forbidden*, *404 Not Found*, *405 Method Not Allowed* y *505 HTTP Version Not Supported*. Además, según el error, el servidor decide si la conexión debe cerrarse o si puede mantenerse abierta.

```
def procesar_error(cs, codigo):
    if codigo == 400:
        mensaje = "Bad Request"
        conexion = "close"
    elif codigo == 403:
        mensaje = "Forbidden"
        conexion = "Keep-Alive"
    elif codigo == 404:
        mensaje = "Not Found"
        conexion = "Keep-Alive"
    elif codigo == 405:
        mensaje = "Method Not Allowed"
        conexion = "Keep-Alive"
    elif codigo == 505:
        mensaje = "HTTP Version Not Supported"
        conexion = "close"

    direccion_imagen = str(codigo) + ".jpg" # Para la imagen del error
    fecha_actual = datetime.now()
    fecha_formateada = fecha_actual.strftime('%Y-%m-%d %H:%M:%S') # El formato del login
    html = "<html><body><h1>Error " + str(codigo) + ": " + mensaje + "</h1> <img src =\"" + direccion_imagen + "\"></body></html>"

    # Construimos la respuesta
    respuesta = "HTTP/1.1 " + str(codigo) + " " + mensaje + "\r\n"
    respuesta += "Server: fontanerosvillanueva6493.org (Ubuntu)\r\n"
    respuesta += "Content-Type: text/html; charset=utf-8\r\n"
    respuesta += "Content-Length: " + str(len(html.encode('utf-8'))) + "\r\n"
    respuesta += "Date: " + fecha_formateada + "\r\n"
    respuesta += "Connection: " + conexion + "\r\n"

    # Si la conexión se mantiene, informamos al cliente
    if conexion == "Keep-Alive":
        respuesta += "Keep-Alive: timeout=" + str(TIMEOUT_CONNECTION) + ", max=100\r\n"
    respuesta += "\r\n"
    respuesta += html

    enviar_mensaje(cs, respuesta)
```

Figura 10: Código de la función *procesar_error*

4. Despliegue del servicio DNS

En este apartado se describe la configuración del servicio DNS de la organización, utilizando el dominio **fontanerosvillanueva6493.org**. Para ello, se ha desplegado un servidor DNS local primario en la máquina servidor y un servidor DNS local secundario en la máquina cliente. De esta forma, ambos pueden resolver los nombres de los equipos y servicios definidos en la organización: **cliente**, **servidor**, **www**, **web**, **smtp**, **pop**, **dns1** y **dns2**.

4.1. Preparación de la resolución local en servidor y en cliente

Primero, se ha revisado el fichero */etc/hosts* en ambas máquinas para asegurar que no existe ninguna referencia manual al dominio, de forma que la resolución de nombres dependa del servicio DNS local y no de entradas estáticas del sistema.

```
alumno@server:~$ cat /etc/hosts
127.0.0.1    localhost
127.0.1.1    server

# The following lines are desirable for IPv6 capable hosts
::1        localhost ip6-localhost ip6-loopback
ff02::1    ip6-allnodes
ff02::2    ip6-allrouters
alumno@server:~$
```

Figura 11: Contenido de */etc/hosts* de la máquina servidor

```

alumno@desktop:~$ cat /etc/hosts
127.0.0.1    localhost
127.0.1.1    desktop

# The following lines are desirable for IPv6 capable hosts
::1         ip6-localhost ip6-loopback
fe00::0     ip6-localnet
ff00::0     ip6-mcastprefix
ff02::1     ip6-allnodes
ff02::2     ip6-allrouters

```

Figura 12: Contenido de `/etc/hosts` de la máquina cliente

Ahora, para que las consultas se envíen al DNS configurado, se ha modificado el fichero `/etc/resolvconf/resolv.conf.d/head` para que la configuración sea persistente, tanto en la máquina cliente como en la máquina servidor, ya que el fichero `/etc/resolv.conf` se regenera automáticamente.

En el servidor se ha configurado el uso del DNS local en 192.168.56.102, mientras que en el cliente se han definido como servidores DNS el primario 192.168.56.102 y el secundario 192.168.56.101. Tras ello, se ejecuta `sudo systemctl restart resolvconf` para que tenga efecto.

```

alumno@server:~$ cat /etc/resolvconf/resolv.conf.d/head
# Dynamic resolv.conf(5) file for glibc resolver(3) generated by resolvconf(8)
#     DO NOT EDIT THIS FILE BY HAND -- YOUR CHANGES WILL BE OVERWRITTEN
nameserver 192.168.56.102

```

Figura 13: Contenido de `/etc/resolvconf/resolv.conf.d/head` de la máquina servidor

```

alumno@desktop:~$ cat /etc/resolvconf/resolv.conf.d/head
# Dynamic resolv.conf(5) file for glibc resolver(3) generated by resolvconf(8)
#     DO NOT EDIT THIS FILE BY HAND -- YOUR CHANGES WILL BE OVERWRITTEN
nameserver 192.168.56.102
nameserver 192.168.56.101

```

Figura 14: Contenido de `/etc/resolvconf/resolv.conf.d/head` de la máquina cliente

4.2. Instalación y configuración del servidor DNS primario

El software utilizado para el servicio DNS es BIND9. La instalación se ha realizado con `sudo apt-get install bind9`. Los ficheros principales modificados han sido `named.conf.options` y `named.conf.local`, ya que son los ficheros para definir las opciones globales del servicio y las zonas gestionadas por el servidor respectivamente. En la máquina servidor, dentro de `named.conf.options`, se ha mantenido el directorio `/var/cache/bind`, se han dejado comentado los `forwarders`, con el objetivo de que, en las consultas a dominios externos, el servidor realice la resolución siguiendo la jerarquía DNS. Por otro lado, se desactiva la directiva `dnssec-validation`, indicando el valor “no”.

```

alumno@server:~$ cat /etc/bind/named.conf.options
options {
    directory "/var/cache/bind";

    // If there is a firewall between you and nameservers you want
    // to talk to, you may need to fix the firewall to allow multiple
    // ports to talk.  See http://www.kb.cert.org/vuls/id/800113

    // If your ISP provided one or more IP addresses for stable
    // nameservers, you probably want to use them as forwarders.
    // Uncomment the following block, and insert the addresses replacing
    // the all-0's placeholder.

    // forwarders {
    //     0.0.0.0;
    // };

    //=====  

    // If BIND logs error messages about the root key being expired,  

    // you will need to update your keys.  See https://www.isc.org/bind-keys  

    //=====  

    dnssec-validation no;

    auth-nxdomain no;    # conform to RFC1035
    listen-on-v6 { any; };
};

```

Figura 15: Contenido de `/etc/bind/named.conf.options` de la máquina servidor

Ahora, en *named.conf.local* se ha definido la zona correspondiente al dominio **fontanerosvillanueva6493.org** como una zona tipo **master**. En esta configuración se indica el fichero de zona */etc/bind/db.fontanerosvillanueva6493.org* y se autoriza la transferencia de zona al servidor secundario. De esta forma, el servidor primario mantiene la autoridad sobre la zona y permite que el secundario obtenga una copia de la misma.

```
alumno@server:~$ cat /etc/bind/named.conf.local
//
// Do any local configuration here
//
// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";
zone "fontanerosvillanueva6493.org" {
    type master;
    file "/etc/bind/db.fontanerosvillanueva6493.org";
    allow-transfer { 192.168.56.101; };
};
```

Figura 16: Contenido de */etc/bind/named.conf.local* de la máquina servidor

4.3. Creación y contenido del fichero de zona

La información del dominio se ha almacenado en el fichero */etc/bind/db.fontanerosvillanueva6493.org*. Este fichero contiene la información concreta del dominio y se apoya en directivas como **\$ORIGIN** y **\$TTL**, además de los registros de recursos asociados a la zona. En él se ha incluido el **registro SOA**, los **registros NS** para los servidores de nombres y los **registros A** necesarios para resolver los principales hosts y servicios. Entre ellos, se han definido los nombres: **cliente**, **servidor**, **www**, **web**, **smtp**, **pop**, **dns1** y **dns2**. También se ha añadido el **registro MX** del dominio, apuntando al servidor de correo *smtp.fontanerosvillanueva6493.org*.

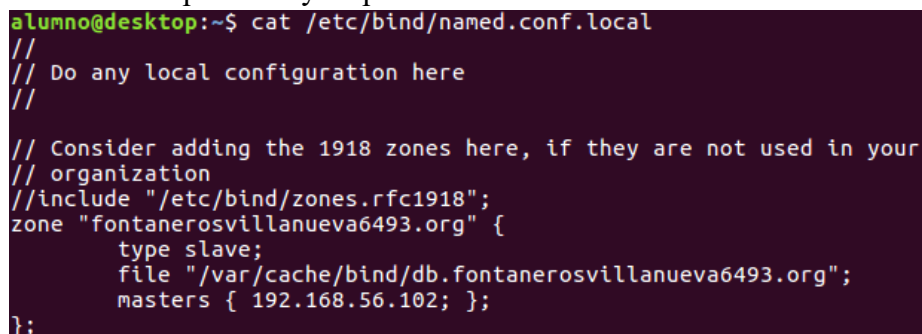
```
alumno@server:~$ cat /etc/bind
bind/
bindresuport.blacklist
alumno@server:~$ cat /etc/bind/db.fontanerosvillanueva6493.org
$TTL 604800
@ IN SOA dns.fontanerosvillanueva6493.org. admin.fontanerosvillanueva6493.org. (
1 ; Serial
604800 ; Refresh
86400 ; Retry
2419200 ; Expire
604800 ) ; Negative Cache TTL
;
;
@ IN NS dns1.fontanerosvillanueva6493.org.
@ IN NS dns2.fontanerosvillanueva6493.org.
;
@ IN MX 10 smtp.fontanerosvillanueva6493.org.
;
dns1 IN A 192.168.56.102
dns2 IN A 192.168.56.101
servidor IN A 192.168.56.102
cliente IN A 192.168.56.101
;
www IN A 192.168.56.102
web IN A 192.168.56.102
smtp IN A 192.168.56.102
pop IN A 192.168.56.102
```

Figura 17: Contenido de */etc/bind/db.fontanerosvillanueva6493.org*

De esta forma, el DNS queda preparado para resolver tanto los nombres de los equipos como los alias de los servicios utilizados.

4.4. Configuración del servidor DNS secundario

En la máquina cliente también se ha instalado **BIND9** para desplegar el servidor DNS secundario con `sudo apt-get install bind9`. En su fichero `named.conf.local` se ha definido la misma zona **fontanerosvillanueva6493.org**, pero en este caso como una zona de tipo **slave**. Además, se ha indicado como maestro la dirección IP del servidor primario **192.168.56.102** y se ha especificado la ruta en la que se almacenará la copia transferida de la zona, en este caso dentro de `/var/cache/bind/`. Así, el secundario puede recibir la información desde el primario y responder a consultas sobre el dominio.



```
alumno@desktop:~$ cat /etc/bind/named.conf.local
//
// Do any local configuration here
//
// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";
zone "fontanerosvillanueva6493.org" {
    type slave;
    file "/var/cache/bind/db.fontanerosvillanueva6493.org";
    masters { 192.168.56.102; };
};
```

Figura 18: Contenido de `/etc/bind/named.conf.local` de la máquina cliente

De esta forma, queda configurado el servicio DNS con un servidor primario, un servidor secundario y una zona capaz de resolver los nombres principales y dar soporte al servicio de correo mediante el registro MX.

5. Despliegue del servicio SMTP/POP

En este apartado se describe la configuración del servicio de correo para la organización, utilizando SMTP para el envío de mensajes y POP3 para su recepción. Para ello, se ha desplegado el servicio en la máquina servidor, utilizando Exim4 como servidor SMTP y Dovecot como servidor POP3. Además, se han creado dos usuarios en el sistema para poder realizar el intercambio de correo y se ha configurado Thunderbird en la máquina cliente para enviar y recibir mensajes utilizando **smtp.fontanerosvillanueva6493.org** y **pop.fontanerosvillanueva6493.org**, que ya habían quedado definidos en la configuración DNS.

5.1. Preparación de la resolución DNS para el servicio de correo

Primero, se ha preparado la resolución de nombres necesaria para el servicio de correo. Para ello, dentro de la configuración DNS ya se definieron las entradas correspondientes a **smtp.fontanerosvillanueva6493.org** y **pop.fontanerosvillanueva6493.org**, ambas asociadas a la dirección IP del servidor: **192.168.56.102**. De esta forma, tanto el servicio SMTP como el servicio POP pueden localizarse mediante nombre dentro del dominio de la organización. Además, dentro del DNS también se configuró el registro MX del dominio apuntando a **smtp.fontanerosvillanueva6493.org**.

Más específicamente, este registro MX permite asociar el dominio **fontanerosvillanueva6493.org** con el servidor encargado de recibir correo para el dominio, que es **smtp.fontanerosvillanueva6493.org**. Aunque en Thunderbird se ha configurado directamente el servidor SMTP por nombre, el registro MX sigue siendo

necesario para poder representar correctamente el funcionamiento real del correo electrónico en el dominio. Entonces, gracias a este registro, si un servidor externo quisiera enviar un mensaje a alguna cuenta **@fontanerosvillanueva6493.org** podría saber qué servidor debe recibir el correo.

5.2. Instalación y configuración del servicio SMTP con Exim4

En la máquina servidor se ha instalado **Exim4** para el servicio SMTP con: *sudo apt-get update*, *sudo apt-get install exim4* y *sudo dpkg-reconfigure exim4-config*.

Tras la instalación, se ha ejecutado la reconfiguración del servicio para definir sus parámetros principales. En esta configuración se ha seleccionado el modo “**Internet Site**”, se ha indicado como nombre del sistema de correo el dominio **fontanerosvillanueva6493.org** y se ha permitido el reenvío para la subred **192.168.56.0/24**. Esta configuración se queda en el fichero **/etc/exim4/update-exim4.conf.conf**

```
alumno@server:~$ cat /etc/exim4/update-exim4.conf.conf
# /etc/exim4/update-exim4.conf.conf
#
# Edit this file and /etc/mailname by hand and execute update-exim4.conf
# yourself or use 'dpkg-reconfigure exim4-config'
#
# Please note that this is _not_ a dpkg-conf file and that automatic changes
# to this file might happen. The code handling this will honor your local
# changes, so this is usually fine, but will break local schemes that mess
# around with multiple versions of the file.
#
# update-exim4.conf uses this file to determine variable values to generate
# exim configuration macros for the configuration file.
#
# Most settings found in here do have corresponding questions in the
# Debconf configuration, but not all of them.
#
# This is a Debian specific file

dc_eximconfig_configtype='internet'
dc_other_hostnames='fontanerosvillanueva6493.org'
dc_local_interfaces=''
dc_readhost=''
dc_relay_domains='fontanerosvillanueva6493.org'
dc_minimaldns='false'
dc_relay_nets='192.168.56.0/24'
dc_smarthost=''
CFILEMODE='644'
dc_use_split_config='false'
dc_hide_mailname=''
dc_mailname_in_oh='true'
dc_localdelivery='maildir_home'
alumno@server:~$
```

Figura 19: Contenido del fichero **/etc/exim4/update-exim4.conf.conf**

Con esta configuración, Exim4 queda preparado para funcionar como servidor SMTP del dominio. Además, la entrega local se ha configurado en formato Maildir, lo que permite que el buzón de cada usuario utilice el mismo formato que utilizará Dovecot para POP3.

5.3. Instalación y configuración del servicio POP3 con Dovecot

Después de configurar SMTP, se ha instalado **Dovecot** como servidor POP3, utilizando el paquete **dovecot-pop3d**: *sudo apt-get install dovecot-pop3d*.

Para configurar Dovecot, primero se ha editado el fichero: */etc/dovecot/conf.d/10-auth.conf*. En este fichero se ha cambiado la directiva: *disable_plaintext_auth = no*.

```
##
## Authentication processes
##

# Disable LOGIN command and all other plaintext authentications unless
# SSL/TLS is used (LOGINDISABLED capability). Note that if the remote IP
# matches the local IP (ie. you're connecting from the same computer), the
# connection is considered secure and plaintext authentication is allowed.
# See also ssl=required setting.
disable_plaintext_auth = no
```

Figura 20: *disable_plaintext_auth* establecido a “no”

Y se ha establecido *auth_mechanisms = plain*

```
# Space separated list of wanted authentication mechanisms:
#   plain login digest-md5 cram-md5 ntlm rpa apop anonymous gssapi otp skey
#   gss-spnego
# NOTE: See also disable_plaintext_auth setting.
auth_mechanisms = plain
```

Figura 21: *auth_mechanisms = plain*

Así, el servidor permite autenticación simple en texto plano, que es el mecanismo que utilizará después Thunderbird al conectarse al servicio.

Después, se ha modificado el fichero */etc/dovecot/conf.d/10-mail.conf*. En este fichero, se ha quitado el comentario de la línea “*mail_location = maildir:~/Maildir*”, indicando que los buzones de correo de los usuarios se almacenarán en su directorio personal con formato Maildir. Esto permite que Dovecot acceda a los mensajes utilizando el mismo formato de almacenamiento configurado en el servicio de correo.

5.4. Creación de usuarios y buzones Maildir

Para el intercambio de correo dentro del dominio de la organización, se han creado en la máquina servidor los usuarios **rafael_64** y **daniel_93**. Al trabajar con usuarios del sistema, cada uno de ellos dispone de su propio directorio personal, donde se almacenará el buzón en formato Maildir.

A partir de estos usuarios, se han definido las dos cuentas de correo rafael_64@fontanerosvillanueva6493.org y daniel_93@fontanerosvillanueva6493.org, que se utilizarán más adelante para el envío y recepción de mensajes en Thunderbird.

5.5. Configuración de Thunderbird

Una vez desplegado el servicio en el servidor, se ha configurado Thunderbird en la máquina cliente para poder utilizar las cuentas de correo creadas.

Para la recepción de correo se ha configurado:

- Nombre del servidor: *pop.fontanerosvillanueva6493.org*
- Puerto: 110
- Seguridad de la conexión: ninguna
- Método de autenticación: contraseña normal
- Nombre de usuario: *rafael_64* o *daniel_93*, según la cuenta.

Para el envío de correo:

- Nombre del servidor: smtp.fontanerosvillanueva6493.org
- Puerto: 25
- Seguridad de la conexión: ninguna
- Método de autenticación: contraseña normal
- Nombre de usuario SMTP: rafael_64 o daniel_93.

Así, queda preparada toda la configuración necesaria para el intercambio de correo, tanto para el envío de mensajes mediante SMTP como su recepción mediante POP3.

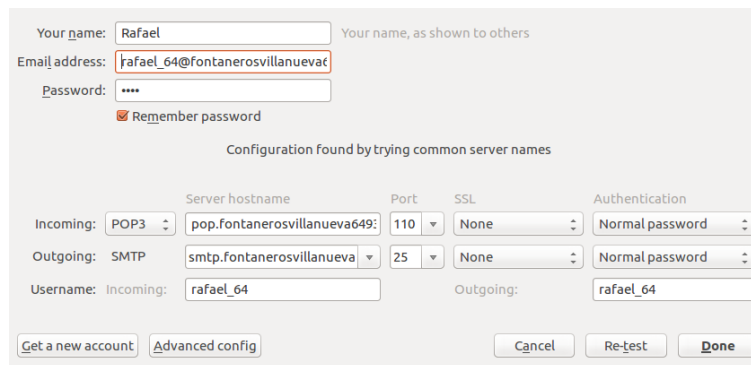


Figura 22: Ejemplo de configuración de Thunderbird para rafael_64

6. Despliegue del servicio HTTP/HTTPS con Apache

6.1. Configuración realizada: HTTP

En este apartado se describe la configuración del servicio web Apache para el acceso mediante HTTP dentro del dominio de la organización. Para ello, se ha desplegado el servicio en la máquina servidor y se ha configurado un VirtualHost asociado al nombre **www.fontanerosvillanueva6493.org**, de forma que el acceso a la página web se realice utilizando dicho nombre dentro del dominio. Además, se ha protegido el acceso al contenido mediante autenticación básica HTTP, solicitando usuario y contraseña antes de mostrar la página.

Primero, se ha preparado la resolución de nombres necesaria para el acceso al servidor web. Para ello, dentro de la configuración DNS ya se había definido la entrada **www.fontanerosvillanueva6493.org**, asociándola a la dirección IP del servidor: **192.168.56.102**. De esta forma, el servicio web puede localizarse por nombre desde la máquina cliente y el acceso al sitio se realiza utilizando la URL correspondiente en lugar de escribir directamente la dirección IP.

Ahora, en la máquina servidor se ha instalado **Apache2** como software servidor web. La instalación se ha realizado mediante: **sudo apt-get update** y después **sudo apt-get install apache2**. Tras esto, se ha creado el directorio raíz del sitio web **/var/www/fontanerosvillanueva6493.org**, que será el DocumentRoot del VirtualHost. Dentro de este directorio se encuentran los ficheros **index.html** y **adicional.html**, que actúan como página principal del servicio web y una página adicional enlazada desde la

principal, respectivamente. Estos ficheros serán los que Apache servirá cuando el cliente acceda a www.fontanerosvillanueva6493.org

```
alumno@server:~$ cat /var/www/fontanerosvillanueva6493.org/index.html
<html>
<head>
    <title>Servidor Simple para Servicios Teleacute:ticos</title>
</head>
<body bgcolor="lightgray">
    <h1>Pagina de Fontaneros Villanueva</h1>

    
    <p>Esta es la pagina web de nuestra organizacion</p>

    <form method="GET">
        Correo electr&oacute;nico:<br>
        <input type="text" name="correo" value=""><br>
        <input type="submit" value="Enviar">
    </form>
    <a href="adicional.html">Ver la pagina adicional</a>
</body>
</html>
```

Figura 23: Contenido de /var/www/fontanerosvillanueva6493.org/index.html

```
alumno@server:~$ cat /var/www/fontanerosvillanueva6493.org/adicional.html
<html>
<head>
    <title>Pagina Adicional de Fontaneros Villanueva</title>
</head>
<body>
    <h1>Autores: Daniel y Rafael</h1>
    <a href="index.html"> <- Volver a la pagina de inicio</a>
</body>
</html>
```

Figura 24: Contenido de /var/www/fontanerosvillanueva6493.org/adicional.html

Después, se ha creado el fichero de configuración del VirtualHost dentro de */etc/apache2/sites-available/*

En este fichero se ha definido el bloque **<VirtualHost *:80>**, indicando el **ServerName** como **www.fontanerosvillanueva6493.org** y el **DocumentRoot** como **/var/www/fontanerosvillanueva6493.org**. Además, dentro del bloque **<Directory>** se ha incluido la configuración para el control de acceso mediante una autenticación básica.

```
alumno@server:~$ cat /etc/apache2/sites-available/fontanerosvillanueva6493.conf
<VirtualHost *:80>
    ServerAdmin admin@fontanerosvillanueva6493.org
    ServerName www.fontanerosvillanueva6493.org
    DocumentRoot /var/www/fontanerosvillanueva6493.org
    <Directory /var/www/fontanerosvillanueva6493.org>
        AllowOverride AuthConfig
        AuthType Basic
        AuthName "Acceso Restringido"
        AuthBasicProvider file
        AuthUserFile /etc/apache2/passwords
        AuthGroupFile /etc/apache2/grupos
        Require group usuariosST
    </Directory>
</VirtualHost>
alumno@server:~$
```

Figura 25: Contenido de /etc/apache2/sites-available/fontanerosvillanueva6493.conf

Para la autenticación, primero se ha creado un fichero de contraseñas usando **htpasswd**. En este fichero se han añadido los usuarios que tendrán permitido el acceso al sitio web. Después, se ha creado manualmente un fichero de grupos, en el que se ha definido el grupo **usuariosST** y se han asociado a él los usuarios autorizados.

```
alumno@server:~$ cat /etc/apache2/grupos
usuariosST: rafael_64 daniel_93
alumno@server:~$
```

Figura 26: Contenido de /etc/apache2/grupos

Ahora, dentro del bloque **Directory** del **VirtualHost** se ha configurado la autenticación básica HTTP. Para ello, se han utilizado las directivas **AuthType Basic**, **AuthName**,

AuthBasicProvider file, AuthUserFile, AuthGroupFile y Require group usuariosST. Así, cuando un cliente intenta acceder al sitio, Apache solicita usuario y contraseña, y solo permite el acceso si el usuario pertenece al grupo autorizado.

Tras esta configuración, se habilita el sitio con *a2ensite*, de forma que quede enlazado dentro de */etc/apache2/sites-enabled/*. Después, se reinicia el servicio Apache para aplicar los cambios realizados.

Finalmente, el acceso al servidor web se realiza mediante la URL <http://www.fontanerosvillanueva6493.org>. Al intentar acceder, Apache solicita las credenciales del usuario antes de mostrar el contenido.

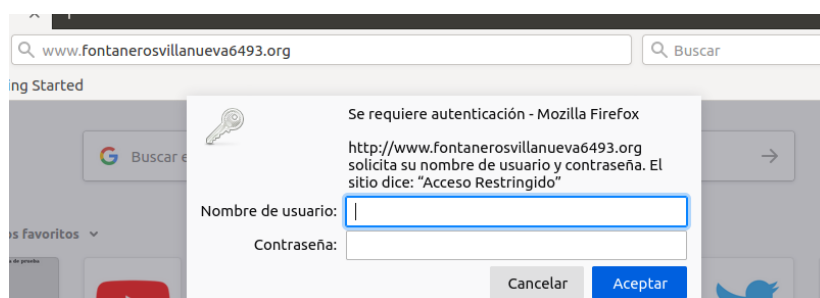


Figura 27: Solicitud de credenciales al intentar acceder



Figura 28: Acceso correcto tras autenticación

6.2. Configuración realizada: HTTPS

En este apartado se describe la configuración del servicio web Apache para el acceso mediante HTTPS dentro del dominio de la organización. Para ello, se ha reutilizado el mismo servicio Apache desplegado en la máquina servidor para HTTP, añadiendo soporte SSL/TLS y utilizando un certificado X.509 de servidor emitido por una Autoridad de Certificación propia. El objetivo es permitir el acceso seguro a <https://www.fontanerosvillanueva6493.org>, de forma que el navegador pueda verificar la identidad del servidor y establecer una conexión cifrada.

6.2.1. Preparación de la PKI y creación de la CA

Primero, se ha preparado la infraestructura de clave pública en el directorio */home/alumno/demoCA*, donde se han creado los ficheros y carpetas necesarias para que OpenSSL pueda actuar como Autoridad de Certificación:

- certs/
- newcerts/

- private/
- index.txt
- serial
- crlnumber

Tras esto, se ha localizado el directorio de configuración de OpenSSL con *openssl version -d* el cual es: */usr/lib/ssl*. En ese directorio se ha modificado el fichero *openssl.cnf* para que la CA trabaje sobre */home/alumno/demoCA*, ajustando también los valores por defecto:

- C = ES
- ST = Murcia
- O = UMU
- OU = fontanerosvillanueva6493.org

Después, se ha generado la Autoridad de Certificación de la organización mediante OpenSSL. La CA se ha generado con clave RSA de 4096 bits en vez de 2048 por seguridad y con una validez de 10 años, obteniendo:

- cacert.pem
- cakey.pem

Entonces el comando utilizado para generar la CA fue:

openssl req -x509 -newkey rsa:4096 -keyout cakey.pem -out cacert.pem -days 3650

Se han introducido los valores identificativos de la CA, quedando esta asociada a:

- C = ES
- ST = Murcia
- O = UMU
- OU = fontanerosvillanueva6493.org
- CN = ca.fontanerosvillanueva6493.org

Entonces, la clave privada se almacena en */home/alumno/demoCA/private/cakey.pem*

6.2.2. Generación del certificado del servidor

Tras crear la CA, se ha generado la clave privada del servidor y su solicitud de certificación, para que luego la CA pudiera firmarla y emitir el certificado del sitio.

Para ello, se ha creado un fichero de configuración adicional: **config.cfg**, utilizado para incluir la extensión SAN y para fijar los valores del certificado del servidor.

```
alumno@server:~/demoCA$ cat config.cfg
[ req ]
distinguished_name = req_distinguished_name
req_extensions = v3_req
prompt = no

[ req_distinguished_name ]
countryName = ES
stateOrProvinceName = Murcia
localityName = Murcia
organizationName = UMU
organizationalUnitName = fontanerosvillanueva6493
commonName = www.fontanerosvillanueva6493.org

[ v3_req ]
subjectAltName = @alt_names

[ alt_names ]
DNS.1 = www.fontanerosvillanueva6493.org
DNS.2 = fontanerosvillanueva6493.org
```

Figura 29: Contenido de config.cfg

A partir de este fichero, se han generado la clave privada y CSR del servidor, con el comando:

```
openssl req -new -nodes -newkey rsa:4096 -keyout serverkey.pem -out servercsr.pem -config config.cfg
```

Como resultado se obtienen: **serverkey.pem** y **servercsr.pem**

Después, la CA ha firmado esta solicitud y ha emitido el certificado final con validez de 2 años, con el comando:

```
openssl ca -keyfile private/cakey.pem -in servercsr.pem -out servercert.pem -days 730
```

Tras esta firma se ha obtenido: **servercert.pem**

El certificado del servidor queda asociado al nombre **www.fontanerosvillanueva6493.org** y además incluye como nombres válidos en el SAN: **www.fontanerosvillanueva6493.org** y **fontanerosvillanueva6493.org**

De esta forma, el certificado queda preparado para ser aceptado por el navegador.

6.2.3. Configuración HTTPS de Apache

Tras generar el certificado del servidor, se ha configurado Apache para aceptar conexiones HTTPS en el puerto 443.

En el fichero **/etc/apache2/ports.conf** se ha dejado habilitada la escucha en el puerto 443 mediante: **Listen 443**.

Después, en el fichero del sitio web:

- **/etc/apache2/sites-available/fontanerosvillanueva6493.conf**

Se ha añadido un nuevo bloque: **<VirtualHost *:443>**. Este bloque reutiliza la misma configuración del sitio HTTP pero se le han añadido las directivas SSL necesarias:

- **SSLEngine on**
- **SSLCertificateFile /home/alumno/demoCA/servercert.pem**
- **SSLCertificateKeyFile /home/alumno/demoCA/serverkey.pem**
- **SSLCACertificateFile /home/alumno/demoCA/cacert.pem**

Con esto, Apache queda preparado para presentar el certificado de servidor al cliente y establecer la conexión HTTPS utilizando la CA propia.

```
<VirtualHost *:443>
    ServerAdmin admin@fontanerosvillanueva6493.org
    ServerName www.fontanerosvillanueva6493.org
    DocumentRoot /var/www/fontanerosvillanueva6493.org
    <Directory /var/www/fontanerosvillanueva6493.org>
        AllowOverride AuthConfig
        AuthType Basic
        AuthName "Acceso Restringido"
        AuthBasicProvider file
        AuthUserFile /etc/apache2/passwords
        AuthGroupFile /etc/apache2/grupos
        Require group usuariosST
    </Directory>
    SSLEngine on
    SSLCertificateFile /home/alumno/demoCA/servercert.pem
    SSLCertificateKeyFile /home/alumno/demoCA/serverkey.pem
    SSLCACertificateFile /home/alumno/demoCA/cacert.pem
</VirtualHost>
```

Figura 30: Bloque <VirtualHost *:443> en fontanerosvillanueva6493.conf

6.2.4. Configuración del navegador cliente para confiar en la CA

Para que el navegador cliente aceptase el certificado del servidor como confiable, primero hay que importar el certificado de la CA. Para ello se copia desde la máquina servidor a la máquina cliente:

```
scp ./cacert.pem alumno@192.168.56.101:/home/alumno
```

Después en Firefox:

Preferences > Advanced > Certificates > View Certificates > Authorities > Import

Y se selecciona *cacert.pem*. Así, Firefox pasó a confiar en la CA, pudiendo verificar correctamente el certificado.

Además, también ha sido necesario cambiar la versión de Firefox utilizada a la 45, ya que la versión 88 produce problemas durante la importación de certificados. Para ello se ejecutaron los siguientes comandos:

- `sudo apt-get remove firefox`
- `sudo apt-get install firefox=45.0.2+build1-0ubuntu1`

6.3. Mejora adicional: Autenticación de cliente mediante certificado

Además de la configuración base de HTTPS, también se ha realizado la mejora de autenticación de cliente mediante certificados X.509. Así, el acceso no dependerá únicamente del certificado del servidor y de la autenticación básica HTTP, sino que se añade un segundo mecanismo de control basado en la identidad criptográfica del usuario.

6.3.1. Generación de las claves privadas y de las CSRs.

Para realizar esta mejora se ha generado la clave privada y la CSR de cada usuario dentro del directorio */home/alumno/demoCA*. Para ello, se han ejecutado los siguientes comandos:

```
openssl req -new -nodes -newkey rsa:4096 -keyout clientkey_rafael.pem -out clientcsr_rafael.pem
```

```
openssl req -new -nodes -newkey rsa:4096 -keyout clientkey_daniel.pem -out clientcsr_daniel.pem
```

Como resultado se obtienen los ficheros:

- **clientkey_rafael.pem**
- **clientcsr_rafael.pem**
- **clientkey_daniel.pem**
- **clientcsr_daniel.pem**

Además, durante la ejecución de este comando se introdujeron los siguientes valores para el certificado del cliente:

- Country Name (C) = ES
- State or Province Name (ST) = Murcia
- Organization Name (O) = UMU
- Organizational Unit Name (OU) = fontanerosvillanueva6493.org
- Common Name (CN) = rafa6493 o daniel6493

Los campos “Email Address”, “Challenge Password” y “Optional Company Name” se dejaron en blanco.

6.3.2. Firma de los certificados por parte de la CA

Una vez generadas las CSR de los clientes, la CA las firma para emitir el certificado final de los usuarios. Para ello se han ejecutado los siguientes comandos:

```
openssl ca -keyfile private/cakey.pem -in clientcsr_rafael.pem -out clientcert_rafael.pem -days 365
```

```
openssl ca -keyfile private/cakey.pem -in clientcsr_daniel.pem -out clientcert_daniel.pem -days 365
```

Como resultado se obtienen los ficheros:

- **clientcert_rafael.pem**
- **clientcert_daniel.pem**

Cada uno de estos ficheros contiene el certificado X.509 de cada usuario, firmado por la misma CA que había emitido el certificado del servidor.

6.3.3. Empaquetado de los certificados en formato PKCS#12

Para poder utilizar los certificados de los clientes desde Firefox, es necesario empaquetar en un único fichero el certificado y la clave privada de cada cliente, y el certificado de la CA. Para ello, se ha generado un fichero en formato PKCS#12, con extensión **.p12**, ejecutando:

```
openssl pkcs12 -export -out rafa6493.p12 -inkey clientkey_rafael.pem -in clientcert_rafael.pem -certfile cacert.pem
```

```
openssl pkcs12 -export -out daniel6493.p12 -inkey clientkey_daniel.pem -in clientcert_daniel.pem -certfile cacert.pem
```

Como resultado se han obtenido:

- **rafa6493.p12**
- **daniel6493.p12**

Estos ficheros contienen toda la información necesaria para que Firefox pueda utilizar el certificado del cliente durante la autenticación HTTPS.

6.3.4. Configuración de Apache para exigir los certificados

Una vez se han generado los certificados, se ha modificado la configuración del sitio HTTPS en Apache para obligar al navegador a presentar un certificado válido durante la conexión. Para ello, dentro del bloque `<VirtualHost *:443>` del fichero `/etc/apache2/sites-available/fontanerosvillanueva6493.conf` se han añadido las siguientes directivas:

- `SSLVerifyClient require`
- `SSLVerifyDepth 1`

Estas directivas se han añadido debajo de los certificados del servidor. El significado de cada una es:

- `SSLVerifyClient require`: obliga al cliente a presentar un certificado válido durante la fase de establecimiento de la conexión SSL/TLS.
- `SSLVerifyDepth 1`: limita la profundidad de la cadena de certificación, exigiendo que el certificado del cliente haya sido firmado directamente por la CA. Se utiliza 1 en vez de 10 como indica la práctica ya que los certificados de cliente están firmados directamente por la CA raíz creada en el servidor.

```
<VirtualHost *:443>
    ServerAdmin admin@fontanerosvillanueva6493.org
    ServerName www.fontanerosvillanueva6493.org
    DocumentRoot /var/www/fontanerosvillanueva6493.org
    <Directory /var/www/fontanerosvillanueva6493.org>
        AllowOverride AuthConfig
        AuthType Basic
        AuthName "Acceso Restringido"
        AuthBasicProvider file
        AuthUserFile /etc/apache2/passwords
        AuthGroupFile /etc/apache2/grupos
        Require group usuariosST
    </Directory>
    SSLEngine on
    SSLCertificateFile /home/alumno/demoCA/servercert.pem
    SSLCertificateKeyFile /home/alumno/demoCA/serverkey.pem
    SSLCACertificateFile /home/alumno/demoCA/cacert.pem
    SSLVerifyClient require
    SSLVerifyDepth 1
</VirtualHost>
```

Figura 31: Bloque `<VirtualHost *:443>` con las nuevas directivas

6.3.5. Importación de los certificados en Firefox

Una vez generados los ficheros .p12 en la máquina servidor, hay que trasladarlos a la máquina cliente para poder importarlos en Firefox. Para ello desde la máquina cliente:

`scp alumno@192.168.56.102:/home/alumno/demoCA/rafael6493.p12 .`

`scp alumno@192.168.56.102:/home/alumno/demoCA/daniel6493.p12 .`

Tras esto, hay que abrir Firefox y:

Preferences > Advanced > Certificates > View Certificates > Your Certificates > Import

Entonces se selecciona el archivo .p12 (rafael6493.p12 o daniel6493.p12) y se introduce la contraseña de exportación definida anteriormente al crear el fichero PKCS#12. Después, se repite para el otro archivo (de rafael o daniel).

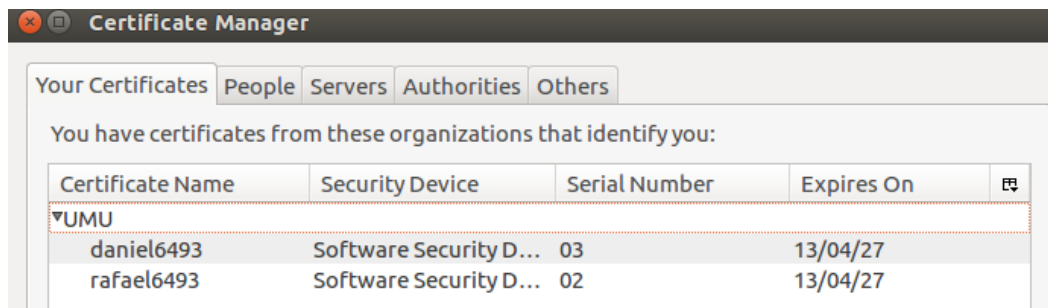


Figura 32: Certificados importados en Firefox

6.3.6. Verificación del acceso con autenticación de cliente

Una vez importados los certificados en Firefox, se ha realizado la prueba de acceso a <https://www.fontanerosvillanueva6493.org>. En este acceso, Firefox ha detectado que el servidor requiere un certificado de cliente y solicita entre utilizar el certificado de Daniel o Rafael, importados en el punto anterior. Tras seleccionar uno, el navegador completa la conexión HTTPS y, una vez validado el certificado del cliente, Apache sigue aplicando control de acceso mediante autenticación básica HTTP.

Por tanto, el acceso final al sitio queda condicionado a dos mecanismos:

1. Presentación de un certificado de cliente válido emitido por la CA

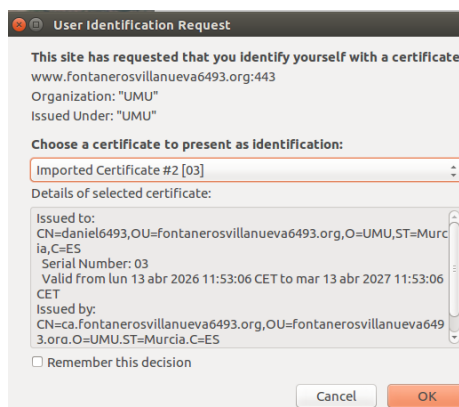


Figura 33: Solicitud del certificado

2. Introducción del usuario y contraseña definidos en Apache para el acceso al contenido

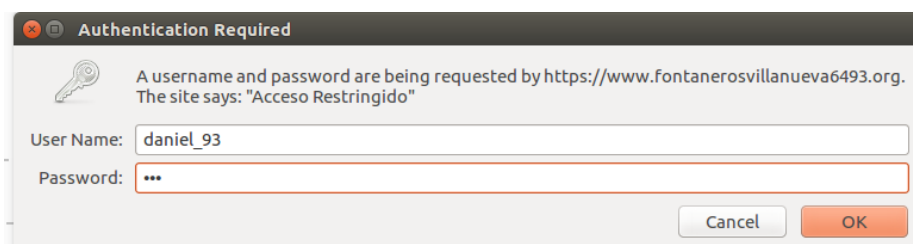


Figura 34: Solicitud de usuario y contraseña

Así, la conexión HTTPS queda protegida mediante doble autenticación: el cliente valida la identidad del servidor mediante su certificado X.509, y el servidor valida la identidad del cliente mediante el certificado de usuario presentado por Firefox



Figura 35: Acceso correcto al sitio

7. Despliegue del servicio IPsec

En este apartado se describe la configuración de IPsec entre la máquina cliente y la máquina servidor, con el objetivo de proteger la comunicación entre ambos equipos. Para ello, se utiliza **strongSwan**, configurando una asociación de seguridad en modo transporte y estableciendo la negociación mediante **certificados X.509**, reutilizando el certificado del servidor web y generando un certificado nuevo para el extremo cliente. Además, la asociación de seguridad se ha configurado mediante **AH**, de forma que la protección aportada se basa en integridad y autenticación del tráfico IP.

7.1. Instalación de strongSwan

Primero, se ha instalado strongSwan en las máquinas cliente y servidor, mediante el comando: ***sudo apt-get install strongswan***

Tras la instalación, se tendrán los dos ficheros principales de configuración que se utilizarán que son:

- /etc/ipsec.conf
- /etc/ipsec.secrets

7.2. Generación del certificado nuevo para el extremo cliente

Dentro de **/home/alumno/demoCA** se ha generado una nueva clave privada y una nueva solicitud de certificación para el extremo cliente de IPsec, mediante el comando:

```
openssl req -new -nodes -newkey rsa:4096 -keyout clientkey_ipsec.pem -out clientcsr_ipsec.pem
```

En esta generación, se han mantenido los mismos valores de la infraestructura de certificados del escenario:

- C = ES
- ST = Murcia
- O = UMU
- OU = fontanerosvillanueva6493.org

Y en el CN se han incluido los nombres y DNIs de los miembros del grupo:

- CN = rafael [REDACTED] daniel [REDACTED]

Después, la solicitud se ha firmado con la CA mediante:

```
openssl ca -keyfile private/cakey.pem -in clientcsr_ipsec.pem -out clientcert_ipsec.pem -days 365
```

Como resultado se obtienen los siguientes ficheros:

- clientkey_ipsec.pem
- clientcsr_ipsec.pem
- clientcert_ipsec.pem

Así, el cliente tendrá un certificado propio firmado por la misma CA que emitió el certificado del servidor web. El certificado del servidor utilizado en IPsec ha sido el **servercert.pem** ya emitido para **www.fontanerosvillanueva6493.org** mientras que para el cliente se ha utilizado este nuevo certificado.

7.3. Copia de certificados a los directorios de strongSwan

Ahora, los certificados se han copiado a los directorios usados por strongSwan. Para ello se han seguido las rutas:

- **/etc/ipsec.d/certs/** para los certificados
- **/etc/ipsec.d/private/** para las claves privadas
- **/etc/ipsec.d/cacert/** para el certificado CA

Entonces, en la máquina servidor se han copiado:

- **servercert.pem** en **/etc/ipsec.d/certs/**
- **serverkey.pem** en **/etc/ipsec.d/private/**
- **cacert.pem** en **/etc/ipsec.d/cacert/**

En la máquina cliente se han copiado:

- **clientcert_ipsec.pem** en **/etc/ipsec.d/certs/**
- **clientkey_ipsec.pem** en **/etc/ipsec.d/private/**
- **cacert.pem** en **/etc/ipsec.d/cacert/**

7.4. Configuración de /etc/ipsec.conf en el servidor

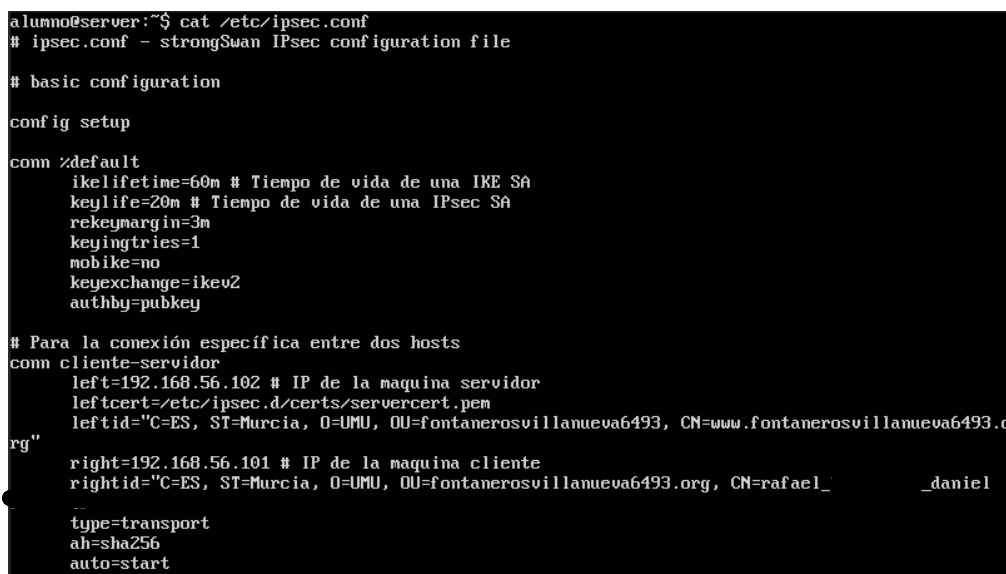
En la máquina servidor se ha configurado el fichero **/etc/ipsec.conf** para definir la conexión IPsec entre 192.168.56.102 y 192.168.56.101.

Primero, dentro del bloque “**conn %default**”, se han definidos los parámetros generales de negociación:

- ikelifetime=60m
- keylife=20m
- rekeymargin=3m
- keyingtries=1
- mobike=no
- keyexchange=ikev2
- authby=pubkey

Así, se fija la negociación mediante IKEv2, el uso de autenticación por certificados y los tiempos de vida de la asociación IKE y de la asociación IPsec. Después, se ha definido una conexión específica llamada **cliente-servidor**, indicando:

- left=192.168.56.102
- leftcert=/etc/ipsec.d/certs/servercert.pem
- leftid="C=ES, ST=Murcia, O=UMU, OU=fontanerosvillanueva6493, CN=www.fontanerosvillanueva6493.org"
- right=192.168.56.101
- rightid="C=ES, ST=Murcia, O=UMU, OU=fontanerosvillanueva6493.org, CN=rafael [REDACTED] daniel [REDACTED]"
- type=transport
- ah=sha256
- auto=start



```
alumno@server:~$ cat /etc/ipsec.conf
# ipsec.conf - strongSwan IPsec configuration file

# basic configuration

config setup
    conn %default
        ikelifetime=60m # Tiempo de vida de una IKE SA
        keylife=20m # Tiempo de vida de una IPsec SA
        rekeymargin=3m
        keyingtries=1
        nobike=no
        keyexchange=ikev2
        authby=pubkey

# Para la conexión específica entre dos hosts
conn cliente-servidor
    left=192.168.56.102 # IP de la maquina servidor
    leftcert=/etc/ipsec.d/certs/servercert.pem
    leftid="C=ES, ST=Murcia, O=UMU, OU=fontanerosvillanueva6493, CN=www.fontanerosvillanueva6493.org"
    right=192.168.56.101 # IP de la maquina cliente
    rightid="C=ES, ST=Murcia, O=UMU, OU=fontanerosvillanueva6493.org, CN=rafael [REDACTED] daniel [REDACTED]"
    type=transport
    ah=sha256
    auto=start
```

Figura 36: Contenido de ipsec.conf en la máquina servidor

El certificado usado por el servidor es **servercert.pem**, que corresponde al mismo certificado X.509 usado para el servicio HTTPS. Además, la asociación se configura con **type=transport**, de forma que se protege directamente el tráfico entre ambos hosts, y con **ah=sha256**, lo que hace que IPsec utilice la cabecera AH con SHA-256 como mecanismo de integridad.

7.5. Configuración de /etc/ipsec.conf en el cliente

La configuración en la máquina cliente es bastante similar a la de la máquina servidor. Dentro del bloque “**conn %default**” se han dejado los mismos parámetros que en el servidor. Después, se ha definido la conexión **cliente-servidor**, indicando:

- left=192.168.56.101
- leftcert=/etc/ipsec.d/certs/clientcert_ipsec.pem
- leftid="C=ES, ST=Murcia, O=UMU, OU=fontanerosvillanueva6493.org, CN=rafael [REDACTED] daniel [REDACTED]"
- right=192.168.56.102

- rightid="C=ES, ST=Murcia, O=UMU, OU=fontanerosvillanueva6493, CN=www.fontanerosvillanueva6493.org"
- type=transport
- ah=sha256
- auto=start

```
alumno@desktop:~$ cat /etc/ipsec.conf
# ipsec.conf - strongSwan IPsec configuration file

# basic configuration

config setup
conn %default
    ikelifetime=60m
    keylife=20m
    rekeymargin=3m
    keyingtries=1
    nobike=no
    keyexchange=ikev2
    authby=pubkey

conn cliente-servidor
    left=192.168.56.101
    leftcert=/etc/ipsec.d/certs/clientcert_ipsec.pem
    leftid="C=ES, ST=Murcia, O=UMU, OU=fontanerosvillanueva6493.org, CN=rafael_daniel_"
    right=192.168.56.102
    rightid="C=ES, ST=Murcia, O=UMU, OU=fontanerosvillanueva6493, CN=www.fontanerosvillanueva6493.org"
    type=transport
    ah=sha256
    auto=start
```

Figura 37: Contenido de ipsec.conf en la máquina cliente

Así, el cliente utiliza como certificado **clientcert_ipsec.pem**, mientras que identifica al extremo servidor mediante el certificado de **www.fontanerosvillanueva6493.org**.

7.6. Configuración de /etc/ipsec.secrets

También ha sido necesario definir en ambos equipos la clave privada asociada al certificado local dentro de **/etc/ipsec.secrets**

En la máquina servidor se ha dejado:

: RSA /etc/ipsec.d/private/serverkey.pem

```
alumno@server:~$ sudo cat /etc/ipsec.secrets
# This file holds shared secrets or RSA private keys for authentication.

# RSA private key for this host, authenticating it to any other host
# which knows the public part.

: RSA /etc/ipsec.d/private/serverkey.pem
```

Figura 38: Contenido de ipsec.secrets en la máquina servidor

En la máquina cliente se ha dejado:

: RSA /etc/ipsec.d/private/clientkey_ipsec.pem

```
alumno@desktop:~$ sudo cat /etc/ipsec.secrets
# This file holds shared secrets or RSA private keys for authentication.

# RSA private key for this host, authenticating it to any other host
# which knows the public part.

: RSA /etc/ipsec.d/private/clientkey_ipsec.pem
```

Figura 39: Contenido de ipsec.secrets en la máquina cliente

Así, strongSwan puede acceder a la clave privada correspondiente a cada certificado durante la autenticación.

7.7. Activación del servicio y comprobación

Una vez completada la configuración, se ha reiniciado strongSwan en ambos equipos con:
sudo ipsec restart

Después, se ha comprobado el estado de la asociación mediante:

- ip x state list

```
alumno@server:~$ sudo ip x state list
src 192.168.56.102 dst 192.168.56.101
  proto ah spi 0xca866e16 reqid 1 mode transport
  replay-window 32
  auth-trunc hmac(sha256) 0x0fcf803bd66a4e4f8fb4f2d6fb9bc0717342593ee9c6dd1ee74ad32bdfb7335 1
28
  anti-replay context: seq 0x0, oseq 0xfb, bitmap 0x00000000
  sel src 192.168.56.102/32 dst 192.168.56.101/32
src 192.168.56.101 dst 192.168.56.102
  proto ah spi 0xc6d05e7e reqid 1 mode transport
  replay-window 32
  auth-trunc hmac(sha256) 0x39867ab1fcff7924877937a0153ea8b7bbd2b4589e38c3db984adfb062567b48 1
28
  anti-replay context: seq 0x10d, oseq 0x0, bitmap 0xffffffff
  sel src 192.168.56.101/32 dst 192.168.56.102/32
```

Figura 40: Salida de ip x state list

- ip x policy list

```
alumno@server:~$ sudo ip x policy list
src 192.168.56.101/32 dst 192.168.56.102/32
  dir in priority 2819
  tmpl src 0.0.0.0 dst 0.0.0.0
  proto ah reqid 1 mode transport
src 192.168.56.102/32 dst 192.168.56.101/32
  dir out priority 2819
  tmpl src 0.0.0.0 dst 0.0.0.0
  proto ah reqid 1 mode transport
```

Figura 41: Salida de ip x policy list

Con esto, se ha podido comprobar que la conexión **cliente-servidor** queda cargada correctamente, que la negociación IKEv2 se completa y que se establece la asociación de seguridad IPsec entre 192.168.56.101 y 192.168.56.102. Además, el tráfico entre ambos equipos queda protegido por AH en modo transporte.

8. Trazas del escenario

En este apartado se recogen varias capturas de Wireshark de los protocolos empleados en el escenario. Para cada una se incluye una explicación del intercambio observado y su relación con los servicios configurados en los anteriores apartados.

8.1. Traza del servicio HTTP desarrollado en Python

En esta traza se puede ver el establecimiento de la conexión TCP entre la máquina cliente y la máquina servidor sobre el puerto 9000. En esta parte se completa el three-way handshake de TCP quedando preparada la conexión para el intercambio de información. Tras establecerse la conexión, se puede observar la solicitud del recurso principal del servicio. Después, el servidor responde con “HTTP/1.1 200 OK”, indicando que la petición ha sido procesada correctamente y que el recurso solicitado existe y se devuelve. Por último, se puede observar el cuerpo HTML de la respuesta, que corresponde a la página principal.

83	11.104877910	192.168.56.101	192.168.56.102	TCP	74	42946 → 9000 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=...
84	11.105017202	192.168.56.102	192.168.56.101	TCP	74	9000 → 42946 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 SACK...
85	11.105636156	192.168.56.101	192.168.56.102	TCP	66	42946 → 9000 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=347549 TSecr=...
86	11.105716404	192.168.56.101	192.168.56.102	HTTP	363	GET / HTTP/1.1
87	11.106191863	192.168.56.102	192.168.56.101	TCP	66	9000 → 42946 [ACK] Seq=1 Ack=298 Win=30080 Len=0 TSval=347200 TSecr=...
88	11.109503522	192.168.56.102	192.168.56.101	TCP	325	9000 → 42946 [PSH, ACK] Seq=1 Ack=298 Win=30080 Len=259 TSval=3472...
89	11.109511279	192.168.56.101	192.168.56.102	TCP	66	42946 → 9000 [ACK] Seq=298 Ack=260 Win=30336 Len=0 TSval=347550 TS...
90	11.110423732	192.168.56.102	192.168.56.101	HTTP	489	HTTP/1.1 200 OK (text/html)
91	11.110430016	192.168.56.101	192.168.56.102	TCP	66	42946 → 9000 [ACK] Seq=298 Ack=683 Win=31360 Len=0 TSval=347550 TS...

• Frame 90: 489 bytes on wire (3912 bits), 489 bytes captured (3912 bits) on interface 0
 • Ethernet II, Src: PcsCompu_d9:a6:a4 (08:00:27:d9:a6:a4), Dst: PcsCompu_b3:3a:20 (08:00:27:b3:3a:20)
 • Internet Protocol Version 4, Src: 192.168.56.101, Dst: 192.168.56.102
 • Transmission Control Protocol, Src Port: 9000, Dst Port: 42946, Seq: 260, Ack: 298, Len: 423
 • [2 Reassembled TCP Segments (682 bytes): #88(259), #90(423)]
 • Hypertext Transfer Protocol
 ▶ HTTP/1.1 200 OK\r\n
 Server: fontanerosvillanueva6493.org (Ubuntu)\r\n
 Content-Type: text/html; charset=utf-8\r\n
 Content-Length: 423\r\n
 Date: 2026-04-26 14:44:14\r\n
 Connection: Keep-Alive\r\n
 Keep-Alive: timeout=32, max=100\r\n
 Set-Cookie: cookie_counter_6493=1; Max-Age=30 \r\n\r\n

Figura 42: Taza de establecimiento de la conexión, solicitud del recurso principal y respuesta del servidor

8.1.1. Taza del formulario GET del servidor

En esta traza se muestra el funcionamiento del formulario incluido en **index.html**. Al introducirse un correo electrónico en el formulario, el navegador genera una petición HTTP con el método GET, añadiendo el valor introducido en la URL.

114	18.317206234	192.168.56.101	192.168.56.102	HTTP	463	GET /?correo=daniel.f.a40um.es HTTP/1.1
115	18.318103626	192.168.56.102	192.168.56.101	HTTP	306	HTTP/1.1 200 OK (text/html)
116	18.318208330	192.168.56.101	192.168.56.102	TCP	66	42946 → 9000 [ACK] Seq=1351 Ack=11034 Win=58496 Len=0 TSval=349352 TSecr=349003
117	18.338590902	192.168.56.101	192.168.56.102	HTTP	375	GET /favicon.ico HTTP/1.1
118	18.340566350	192.168.56.102	192.168.56.101	HTTP	359	HTTP/1.1 404 Not Found (text/html)
119	18.380051321	192.168.56.101	192.168.56.102	TCP	66	42946 → 9000 [ACK] Seq=1600 Ack=11327 Win=61440 Len=0 TSval=349368 TSecr=349008
120	19.939015417	192.168.56.101	192.168.56.102	HTTP	375	GET /favicon.ico HTTP/1.1
121	19.938622604	192.168.56.102	192.168.56.101	HTTP	359	HTTP/1.1 404 Not Found (text/html)
122	19.938643187	192.168.56.101	192.168.56.102	TCP	66	42946 → 9000 [ACK] Seq=1069 Ack=11620 Win=64384 Len=0 TSval=349757 TSecr=349407
123	20.31747317	192.168.56.101	192.168.56.102	HTTP	430	GET /?correo=daniel.f.a40um.es HTTP/1.1
124	20.34116400	192.168.56.102	192.168.56.101	HTTP	206	HTTP/1.1 200 OK (text/html)

• Frame 123: 458 bytes on wire (3664 bits), 458 bytes captured (3664 bits) on interface 0
 • Ethernet II, Src: PcsCompu_b3:3a:20 (08:00:27:b3:3a:20), Dst: PcsCompu_d9:a6:a4 (08:00:27:d9:a6:a4)
 • Internet Protocol Version 4, Src: 192.168.56.101, Dst: 192.168.56.102
 • Transmission Control Protocol, Src Port: 42946, Dst Port: 9000, Seq: 1969, Ack: 11620, Len: 392
 • Hypertext Transfer Protocol
 ▶ GET /?correo=daniel.f.a40um.es HTTP/1.1\r\n
 Host: 192.168.56.102:9000\r\n
 User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:45.0) Gecko/20100101 Firefox/45.0\r\n
 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
 Accept-Language: en-US,en;q=0.5\r\n
 Accept-Encoding: gzip, deflate\r\n
 Referer: http://192.168.56.102:9000/\r\n
 Cookie: cookie_counter_6493=2\r\n
 Connection: keep-alive\r\n\r\n
 [Full request URI: http://192.168.56.102:9000/?correo=daniel.f.a40um.es]
 [HTTP request 7/33]
 [Prev request in frame: 120]
 [Response in frame: 124]
 [Next request in frame: 126]

Figura 43: Taza del formulario GET

8.1.2. Taza de cookies y bloqueo de acceso

En esta traza se observa la gestión de cookies implementada. El valor de la cookie **cookie_counter_6493** se incrementa para controlar el número de accesos del cliente. En las respuestas del servidor aparece la cabecera **Set-Cookie**, con la que se envía al navegador el contador actualizado. En las siguientes peticiones, el navegador vuelve a enviar la cookie al servidor mediante la cabecera **Cookie**, para que así el servidor pueda conocer el valor anterior y actualizarlo. Si se alcanza el máximo permitido, el servidor deja de devolver el contenido solicitado y responde con **403 Forbidden**, bloqueando temporalmente el acceso.

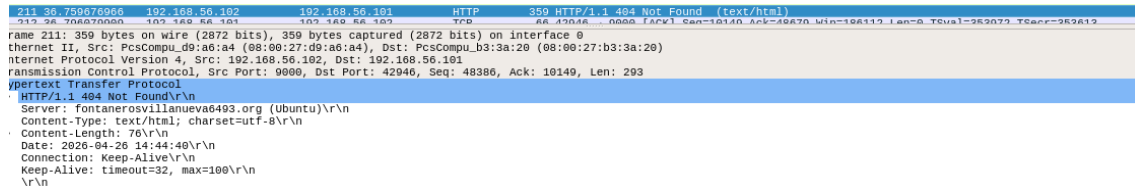
100	36.467122424	192.168.56.102	192.168.56.101	HTTP	480	HTTP/1.1 200 OK (text/html)
101	36.467125175	192.168.56.101	192.168.56.102	TCP	66	42946 → 9000 [ACK] Seq=7455 Ack=46335 Win=183040 Len=0 TSval=353889 TSecr=353540
102	36.474181067	192.168.56.101	192.168.56.102	HTTP	430	GET /logo-um.jpg HTTP/1.1
103	36.476356092	192.168.56.102	192.168.56.101	HTTP	359	HTTP/1.1 403 Forbidden (text/html)
104	36.492813104	192.168.56.101	192.168.56.102	HTTP	375	GET /favicon.ico HTTP/1.1

Transmission Control Protocol, Src Port: 9000, Dst Port: 42946, Seq: 45912, Ack: 7455, Len: 423
 [2 Reassembled TCP Segments (682 bytes): #188(259), #190(423)]
 Hypertext Transfer Protocol
 ▶ HTTP/1.1 200 OK\r\n
 Server: fontanerosvillanueva6493.org (Ubuntu)\r\n
 Content-Type: text/html; charset=utf-8\r\n
 Content-Length: 423\r\n
 Date: 2026-04-26 14:44:40\r\n
 Connection: Keep-Alive\r\n
 Keep-Alive: timeout=32, max=100\r\n
 Set-Cookie: cookie_counter_6493=9; Max-Age=30 \r\n\r\n
 [HTTP response 22/33]

Figura 44: Taza de gestión de cookies y respuesta 403 Forbidden

8.1.3. Trazas de solicitud de recurso inexistente

En esta traza se puede observar una petición a un recurso inexistente **/favicon.ico** (podría ser cualquier otro fichero que no sirva el servidor). Al no encontrar el recurso, el servidor responde con **HTTP/1.1 404 Not Found**.



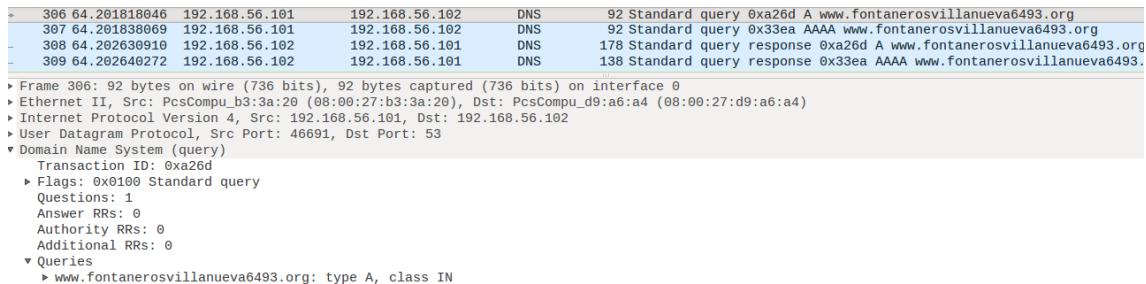
No.	Time	Source	Destination	Protocol	Length	Info
211	36.759676986	192.168.56.102	192.168.56.101	HTTP	359	HTTP/1.1 404 Not Found (text/html)

Frame 211: 359 bytes on wire (2872 bits), 359 bytes captured (2872 bits) on interface 0
Ethernet II, Src: PcsCompu_d9:a6:a4 (08:00:27:d9:a6:a4), Dst: PcsCompu_b3:3a:20 (08:00:27:b3:3a:20)
Internet Protocol Version 4, Src: 192.168.56.102, Dst: 192.168.56.101
Transmission Control Protocol, Src Port: 9000, Dst Port: 42946, Seq: 48386, Ack: 10149, Len: 293
Hypertext Transfer Protocol
Server: fontanerosvillanueva6493.org (Ubuntu)\r\n
Content-Type: text/html; charset=utf-8\r\n
Content-Length: 76\r\n
Date: 2026-04-26 14:44:40\r\n
Connection: Keep-Alive\r\n
Keep-Alive: timeout=32, max=100\r\n
\r\n

Figura 45: Trazas de respuesta del servidor tras una solicitud de un recurso inexistente

8.2. Trazas DNS y acceso a <http://www.fontanerosvillanueva6493.org>

En esta traza se puede observar primero la resolución DNS necesaria para acceder al servidor web mediante nombre de dominio. Se puede ver que la máquina cliente consulta por **www.fontanerosvillanueva6493.org**, realizándose la resolución del nombre antes de intentar acceder al servicio HTTP.

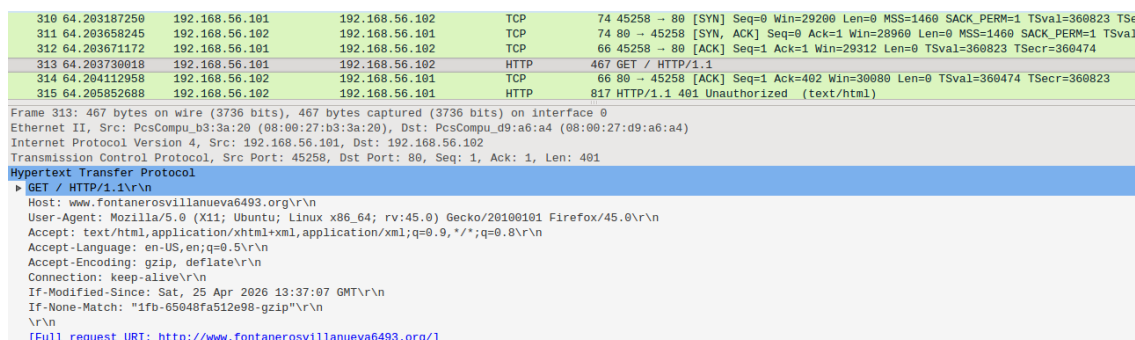


No.	Time	Source	Destination	Protocol	Length	Info
396	64.201818046	192.168.56.101	192.168.56.102	DNS	92	Standard query 0xa26d A www.fontanerosvillanueva6493.org
397	64.201838069	192.168.56.101	192.168.56.102	DNS	92	Standard query 0x33ea AAAA www.fontanerosvillanueva6493.org
398	64.202630910	192.168.56.102	192.168.56.101	DNS	178	Standard query response 0xa26d A www.fontanerosvillanueva6493.org
399	64.202640272	192.168.56.102	192.168.56.101	DNS	138	Standard query response 0x33ea AAAA www.fontanerosvillanueva6493.org

Frame 396: 92 bytes on wire (736 bits), 92 bytes captured (736 bits) on interface 0
Ethernet II, Src: PcsCompu_b3:3a:20 (08:00:27:b3:3a:20), Dst: PcsCompu_d9:a6:a4 (08:00:27:d9:a6:a4)
Internet Protocol Version 4, Src: 192.168.56.101, Dst: 192.168.56.102
User Datagram Protocol, Src Port: 46691, Dst Port: 53
Domain Name System (query)
Transaction ID: 0xa26d
Flags: 0x0100 Standard query
Questions: 1
Answer RRs: 0
Authority RRs: 0
Additional RRs: 0
Queries
www.fontanerosvillanueva6493.org: type A, class IN

Figura 46: Trazas de Wireshark de la resolución DNS

Una vez resuelto el nombre, el cliente establece conexión con el puerto 80 del servidor. Primero se observa un acceso HTTP que obtiene como respuesta **“401 Unauthorized”**. Esto indica que el servidor funciona correctamente, pero que el recurso solicitado está protegido mediante autenticación básica, por lo que exige unas credenciales válidas antes de permitir el acceso.



No.	Time	Source	Destination	Protocol	Length	Info
310	64.203187250	192.168.56.101	192.168.56.102	TCP	74	45258 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=360823 TSecr=360823
311	64.203658245	192.168.56.102	192.168.56.101	TCP	74	80 → 45258 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 SACK_PERM=1 TSval=360823 TSecr=360823
312	64.203671172	192.168.56.101	192.168.56.102	TCP	66	45258 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=360823 TSecr=360823
313	64.203730018	192.168.56.101	192.168.56.102	HTTP	467	GET / HTTP/1.1
314	64.204112958	192.168.56.102	192.168.56.101	TCP	66	80 → 45258 [ACK] Seq=1 Ack=402 Win=30000 Len=0 TSval=360823 TSecr=360823
315	64.205852688	192.168.56.102	192.168.56.101	HTTP	817	HTTP/1.1 401 Unauthorized (text/html)

Frame 313: 467 bytes on wire (3736 bits), 467 bytes captured (3736 bits) on interface 0
Ethernet II, Src: PcsCompu_b3:3a:20 (08:00:27:b3:3a:20), Dst: PcsCompu_d9:a6:a4 (08:00:27:d9:a6:a4)
Internet Protocol Version 4, Src: 192.168.56.101, Dst: 192.168.56.102
Transmission Control Protocol, Src Port: 45258, Dst Port: 80, Seq: 1, Ack: 1, Len: 401
Hypertext Transfer Protocol
GET / HTTP/1.1\r\n
Host: www.fontanerosvillanueva6493.org\r\n
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:45.0) Gecko/20100101 Firefox/45.0\r\n
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
Accept-Language: en-US;q=0.5\r\n
Accept-Encoding: gzip, deflate\r\n
Connection: keep-alive\r\n
If-Modified-Since: Sat, 25 Apr 2026 13:37:07 GMT\r\n
If-None-Match: "1fb-65048fa512e08-gzip"\r\n
\r\n
[Full request URI: <http://www.fontanerosvillanueva6493.org/>]

Figura 47: Trazas de intento de acceso sin autenticar

320	70.024818523	192.168.56.101	192.168.56.102	TCP	74	45262 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=3622779 TSecr=361929
321	70.025163179	192.168.56.102	192.168.56.101	TCP	74	80 → 45262 [SYN, ACK] Seq=0 Ack=1 Wln=28960 Len=0 MSS=1460 SACK_PERM=1 TSval=3622779 TSecr=361929
322	70.025178687	192.168.56.101	192.168.56.102	TCP	66	45262 → 80 [ACK] Seq=1 Ack=1 Wln=29312 Len=0 TSval=3622779 TSecr=361929
323	70.025247308	192.168.56.101	192.168.56.102	HTTP	510	GET / HTTP/1.1
324	70.025445939	192.168.56.102	192.168.56.101	TCP	66	80 → 45262 [ACK] Seq=1 Ack=445 Wln=30080 Len=0 TSval=361929 TSecr=362279
325	70.027680777	192.168.56.102	192.168.56.101	HTTP	715	HTTP/1.1 200 OK (text/html)

Figura 48: Respuesta del servidor con el recurso principal tras autenticación correcta

En esta traza se observa el acceso mediante <https://www.fontanerosvillanueva6493.org>. En este caso no aparece una consulta DNS previa ya que el nombre del servidor ya había sido resuelto durante el acceso por HTTP en el apartado anterior. Por esto, el cliente reutiliza la información de resolución que tiene disponible y establece directamente la conexión con el puerto 443 del servidor apache.

Figura 49: Traza de Wireshark del establecimiento de conexión del cliente con el puerto 443, solicitud del certificado al cliente y envío del certificado por el cliente.

Autores: Rafael Guillén García y Daniel Fernández Ayala.

8.4. Traza DNS, SMTP y POP

8.4.1. DNS y SMTP

En esta traza se observa primero la resolución DNS necesaria para localizar el servidor SMTP del dominio ya que, antes de enviar el mensaje, el cliente necesita conocer la dirección IP asociada a **smtp.fontanerosvillanueva6493.org**.

559	143.296386819	192.168.56.101	192.168.56.102	DNS	93 Standard query 0x8669 A smtp.fontanerosvillanueva6493.org
560	143.296409478	192.168.56.101	192.168.56.102	DNS	93 Standard query 0x5bb6 AAAA smtp.fontanerosvillanueva6493.org
561	143.296970076	192.168.56.102	192.168.56.101	DNS	179 Standard query response 0x8669 A smtp.fontanerosvillanueva6493.org A 192.168.
562	143.296983193	192.168.56.102	192.168.56.101	DNS	139 Standard query response 0x5bb6 AAAA smtp.fontanerosvillanueva6493.org SOA dn.
563	143.297070297	192.168.56.101	192.168.56.102	TCP	74 48006 → 25 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=380597 TSecr=
564	143.297193811	192.168.56.102	192.168.56.101	TCP	74 25 → 48006 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 SACK_PERM=1 TSval=
565	143.297204923	192.168.56.101	192.168.56.102	TCP	66 48006 → 25 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=380597 TSecr=380247
566	143.298426469	192.168.56.102	192.168.56.101	SMTP	135 S: 250 server ESMTP Exim 4.86_2 Ubuntu Sun, 26 Apr 2026 14:46:26 +0200
567	143.298434702	192.168.56.101	192.168.56.102	TCP	66 48006 → 25 [ACK] Seq=1 Ack=70 Win=29312 Len=0 TSval=380597 TSecr=380248
568	143.318736995	192.168.56.101	192.168.56.102	SMTP	89 C: EHLO [192.168.56.101]
569	143.319094199	192.168.56.102	192.168.56.101	TCP	66 25 → 48006 [ACK] Seq=70 Ack=24 Win=29056 Len=0 TSval=380253 TSecr=380602
570	143.319314486	192.168.56.102	192.168.56.101	SMTP	187 S: 250-server Hello [192.168.56.101] [192.168.56.101] 250-SIZE 52428800
571	143.371552834	192.168.56.101	192.168.56.102	TCP	66 48006 → 25 [ACK] Seq=24 Ack=191 Win=29312 Len=0 TSval=380615 TSecr=380253
572	143.385438556	192.168.56.101	192.168.56.102	SMTP	141 C: MAIL FROM:<rafael_64@fontanerosvillanueva6493.org> BODY=8BITIME SIZE=481
573	143.387233953	192.168.56.102	192.168.56.101	SMTP	74 S: 250 OK
574	143.387251153	192.168.56.101	192.168.56.102	TCP	66 48006 → 25 [ACK] Seq=99 Ack=199 Win=29312 Len=0 TSval=380619 TSecr=380270
575	143.388360434	192.168.56.101	192.168.56.102	SMTP	116 C: RCPT TO:<daniel_93@fontanerosvillanueva6493.org>
576	143.388927425	192.168.56.102	192.168.56.101	SMTP	89 S: 250 Accepted
577	143.392653631	192.168.56.101	192.168.56.102	SMTP	72 C: DATA
578	143.393108753	192.168.56.102	192.168.56.101	SMTP	122 S: 354 Enter message, ending with "." on a line by itself
579	143.393881210	192.168.56.101	192.168.56.102	SMTP	547 C: DATA fragment, 481 bytes
580	143.393971867	192.168.56.101	192.168.56.102	SMTP IMF	69 from: rafael_64 <rafael_64@fontanerosvillanueva6493.org>, subject: PRUEBA,
581	143.394319218	192.168.56.102	192.168.56.101	TCP	66 25 → 48006 [ACK] Seq=269 Ack=639 Win=30000 Len=0 TSval=380272 TSecr=380621
582	143.405410478	192.168.56.102	192.168.56.101	SMTP	94 S: 250 OK id=1wGysU-0000X2-Vt

▶ Frame 559: 93 bytes on wire (744 bits), 93 bytes captured (744 bits) on interface 0
▶ Ethernet II, Src: PcsCompu_b3:3a:20 (08:00:27:b3:3a:20), Dst: PcsCompu_d9:a6:a4 (08:00:27:d9:a6:a4)
▶ Internet Protocol Version 4, Src: 192.168.56.101, Dst: 192.168.56.102
▶ User Datagram Protocol, Src Port: 49993, Dst Port: 53
▼ Domain Name System (query)
Transaction ID: 0x8669
Flags: 0x0100 Standard query
Questions: 1
Answer RRs: 0
Authority RRs: 0
Additional RRs: 0
▼ Queries
▼ smtp.fontanerosvillanueva6493.org: type A, class IN
Name: smtp.fontanerosvillanueva6493.org

Figura 50: Traza de Wireshark con la resolución DNS de **smtp.fontanerosvillanueva6493.org**, establecimiento de conexión y envío de correo

Después de esta resolución, se establece la conexión con el puerto 25, que corresponde al servicio SMTP. Tras esto, aparece el diálogo SMTP completo entre el cliente y el servidor de correo, donde se pueden observar comandos como:

- **EHLO**: comando mediante el cual el cliente se identifica ante el servidor e inicia la sesión SMTP
- **MAIL FROM**: indica el remitente del mensaje
- **RCPT TO**: especifica el destinatario
- **DATA**: marca el comienzo del contenido del correo electrónico

Las respuestas **250 OK** devueltas por el servidor en distintos momentos indican que el servidor Exim4 acepta correctamente cada fase del envío.

8.4.2. DNS y POP

En esta traza se observa la resolución DNS del nombre **pop.fontanerosvillanueva6493.org** que es necesaria para que el cliente pueda localizar el servidor POP3 del dominio. Tras obtener la dirección IP, el cliente establece la conexión con el puerto 110.

Después, se observa el proceso de autenticación del cliente y varias operaciones del protocolo POP, como:

- **STAT**: utilizado para consultar el estado del buzón
- **LIST**: para obtener el listado de mensajes
- **UIDL**: para consultar los identificadores únicos de los correos

- **RETR 8:** operación mediante la cual el cliente solicita la recuperación de un mensaje concreto del buzón.

590	146.729432471	192.168.56.101	192.168.56.102	DNS	92 Standard query 0xea32 A pop.fontanerosvillanueva6493.org
591	146.729454999	192.168.56.101	192.168.56.102	DNS	92 Standard query 0x47b9 AAAA pop.fontanerosvillanueva6493.org
592	146.730185107	192.168.56.102	192.168.56.101	DNS	178 Standard query response 0xea32 A pop.fontanerosvillanueva6493.org A 192.168...
593	146.730198264	192.168.56.102	192.168.56.101	DNS	138 Standard query response 0x47b9 AAAA pop.fontanerosvillanueva6493.org SOA dns...
594	146.730334058	192.168.56.101	192.168.56.102	TCP	74 39064 → 110 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=381455 TS...
595	146.730823431	192.168.56.102	192.168.56.101	TCP	74 110 → 39064 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 SACK_PERM=1 TSva...
596	146.730846274	192.168.56.101	192.168.56.102	TCP	66 39064 → 110 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=381455 TSecr=381106
597	146.733405519	192.168.56.102	192.168.56.101	POP	86 S: +OK Dovecot ready.
598	146.733426442	192.168.56.101	192.168.56.102	TCP	66 39064 → 110 [ACK] Seq=1 Ack=21 Win=29312 Len=0 TSval=381456 TSecr=381106
599	146.733547042	192.168.56.101	192.168.56.102	POP	72 C: CAPA
600	146.737990308	192.168.56.102	192.168.56.101	TCP	66 110 → 39064 [ACK] Seq=21 Ack=7 Win=29056 Len=0 TSval=381107 TSecr=381456
601	146.738042226	192.168.56.102	192.168.56.101	POP	149 S: +OK
602	146.7363711672	192.168.56.101	192.168.56.102	POP	78 C: AUTH PLAIN
603	146.736711590	192.168.56.102	192.168.56.101	POP IMF	70 +
604	146.736771020	192.168.56.101	192.168.56.102	POP	88 C: AGRhbm1lbF85MwAzMjE=
605	146.743496922	192.168.56.102	192.168.56.101	POP	82 S: +OK Logged in.
606	146.743581331	192.168.56.101	192.168.56.102	POP	72 C: STAT
607	146.770123209	192.168.56.102	192.168.56.101	POP	78 S: +OK 8 6908
608	146.770943668	192.168.56.101	192.168.56.102	POP	72 C: LIST
609	146.771352735	192.168.56.102	192.168.56.101	POP	142 S: +OK 8 messages:
610	146.771485903	192.168.56.101	192.168.56.102	POP	72 C: UIDL
611	146.772089899	192.168.56.102	192.168.56.101	POP	234 S: +OK
612	146.772466519	192.168.56.101	192.168.56.102	POP	74 C: RETR 8
613	146.772924061	192.168.56.102	192.168.56.101	POP	951 S: +OK 866 octets

Frame 590: 92 bytes on wire (736 bits), 92 bytes captured (736 bits) on interface 0
 Internet II, Src: PcsCompu_b3:3a:20 (08:00:27:b3:3a:20), Dst: PcsCompu_d9:a6:a4 (08:00:27:d9:a6:a4)
 Internet Protocol Version 4, Src: 192.168.56.101, Dst: 192.168.56.102
 User Datagram Protocol, Src Port: 36738, Dst Port: 53
 Domain Name System (query)
 Transaction ID: 0xea32
 Flags: 0x0100 Standard query
 Questions: 1
 Answer RRs: 0
 Authority RRs: 0
 Additional RRs: 0
 Queries
 pop.fontanerosvillanueva6493.org: type A, class IN
 Name: pop.fontanerosvillanueva6493.org

Figura 51: Traza de Wireshark con la resolución DNS de pop.fontanerosvillanueva6493.org, establecimiento de la conexión TCP y operaciones de POP

8.4.3. Consulta al registro MX

En esta traza se puede observar la consulta al registro MX del dominio. Con esta traza, se puede observar qué servidor se encarga de recibir correo para el dominio.

848	1117369.7630131	192.168.56.101	192.168.56.102	DNS	99 Standard query 0x4497 MX fontanerosvillanueva6493.org OPT
849	1117369.7644564	192.168.56.102	192.168.56.101	DNS	206 Standard query response 0x4497 MX fontanerosvillanueva6493.org...

Domain Name System (response)
 Transaction ID: 0x4497
 Flags: 0x8580 Standard query response, No error
 Questions: 1
 Answer RRs: 1
 Authority RRs: 2
 Additional RRs: 4
 Queries
 Answers
 fontanerosvillanueva6493.org: type MX, class IN, preference 10, mx smtp.fontanerosvillanueva6493.org
 Authoritative nameservers
 fontanerosvillanueva6493.org: type NS, class IN, ns dns1.fontanerosvillanueva6493.org
 fontanerosvillanueva6493.org: type NS, class IN, ns dns2.fontanerosvillanueva6493.org
 Additional records
 smtp.fontanerosvillanueva6493.org: type A, class IN, addr 192.168.56.102
 dns1.fontanerosvillanueva6493.org: type A, class IN, addr 192.168.56.102
 dns2.fontanerosvillanueva6493.org: type A, class IN, addr 192.168.56.101
 <Root>: type OPT

Figura 52: Traza de Wireshark de la consulta al registro MX

La respuesta DNS contiene el registro MX configurado, apuntando a **smtp.fontanerosvillanueva6493.org**.

8.5. IKE y IPsec

En esta traza se puede observar el establecimiento de la asociación de seguridad IPsec entre la máquina cliente y la máquina servidor. Los paquetes 631 y 632 corresponden al intercambio “IKE_SA_INIT” que es la fase inicial de la negociación IKEv2. En esta fase los dos extremos intercambian la información necesaria para acordar los parámetros de criptográficos que se utilizarán en la comunicación protegida.

631	175.996077431	192.168.56.101	192.168.56.102	ISAKMP	1166 IKE_SA_INIT MID=00 Initiator Request
632	176.003288294	192.168.56.102	192.168.56.101	ISAKMP	523 IKE_SA_INIT MID=00 Responder Response
633	176.019706225	192.168.56.101	192.168.56.102	IPv4	1514 Fragmented IP protocol (proto=UDP 17, off=0, ID=49b6) [Reassembled in #634]
634	176.019723969	192.168.56.102	192.168.56.101	ISAKMP	1134 IKE_AUTH MID=01 Initiator Request
635	176.026206676	192.168.56.101	192.168.56.102	IPv4	1514 Fragmented IP protocol (proto=UDP 17, off=0, ID=73c6) [Reassembled in #636]
636	176.026300322	192.168.56.102	192.168.56.101	ISAKMP	878 IKE_AUTH MID=01 Responder Response

Figura 53: Traza de Wireshark con el establecimiento de la asociación de seguridad de IPsec, el intercambio IKE_SA_INIT y la fase IKE_AUTH

En los paquetes 634 y 636 aparece la fase “IKE_AUTH”, en la que se completa la autenticación entre ambos extremos. Esta autenticación se realiza mediante certificados X.509, utilizando el certificado del servidor web en un extremo y el certificado nuevo generado específicamente para IPsec en el otro.

Tras establecerse la asociación de seguridad, se puede observar en las siguientes imágenes los paquetes protegidos mediante AH:

981	1117414.2599793..	192.168.56.101	192.168.56.102	HTTP	542 GET / HTTP/1.1
982	1117414.2637337..	192.168.56.102	192.168.56.101	HTTP	746 HTTP/1.1 200 OK (text/html)
983	1117414.2637952..	192.168.56.101	192.168.56.102	TCP	98 47618 - 80 [ACK] Seq=846 Ack=1408 Win=32256 Len=0 TSval=4294
984	1117414.3935563..	192.168.56.101	192.168.56.102	HTTP	585 GET /fontaneros-villanueva.png HTTP/1.1
985	1117414.3952931..	192.168.56.102	192.168.56.101	HTTP	289 HTTP/1.1 304 Not Modified
986	1117414.3953907..	192.168.56.101	192.168.56.102	TCP	98 47618 - 80 [ACK] Seq=1333 Ack=1582 Win=33792 Len=0 TSval=429
Frame 981: 542 bytes on wire (4336 bits), 542 bytes captured (4336 bits) on interface 0					
Ethernet II, Src: PcsCompu_b3:3a:20 (08:00:27:b3:3a:20), Dst: PcsCompu_09:a6:a4 (08:00:27:d9:a6:a4)					
Internet Protocol Version 4, Src: 192.168.56.101, Dst: 192.168.56.102					
Authentication Header					
Next header: TCP (6)					
Length: 6 (32 bytes)					
Reserved: 0000					
AH SPI: 0xc0fdedab					
AH Sequence: 67					
AH ICV: add021284e2d777fb5582510ad6e9a2c0a83866					

Figura 54: Bloque HTTP protegidos por AH

747	188.268194438..	192.168.56.101	192.168.56.102	TCP	106 44118 - 443 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1
748	188.268713936..	192.168.56.102	192.168.56.101	TCP	106 443 - 44118 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 S
749	188.268743886..	192.168.56.101	192.168.56.102	TCP	98 44118 - 443 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=396184 TS
750	188.268878941..	192.168.56.101	192.168.56.102	TLSv1.2	384 Client Hello
751	188.269014984..	192.168.56.102	192.168.56.101	TCP	98 443 - 44118 [ACK] Seq=1 Ack=207 Win=30800 Len=0 TSval=391490
752	188.277520299..	192.168.56.102	192.168.56.101	TLSv1.2	1514 Server Hello
753	188.277568184..	192.168.56.101	192.168.56.102	TCP	98 44118 - 443 [ACK] Seq=207 Ack=1417 Win=32128 Len=0 TSval=39118
754	188.277580062..	192.168.56.102	192.168.56.101	TCP	1514 443 - 44118 [ACK] Seq=1417 Ack=207 Win=30800 Len=1416 TSval=3
755	188.277595398..	192.168.56.101	192.168.56.102	TCP	98 44118 - 443 [ACK] Seq=207 Ack=2833 Win=35072 Len=0 TSval=39118
756	188.277602473..	192.168.56.102	192.168.56.101	TLSv1.2	1187 Certificate, Server key Exchange, Certificate Request, Server
757	188.277616228..	192.168.56.101	192.168.56.102	TCP	98 44118 - 443 [ACK] Seq=207 Ack=3842 Win=37888 Len=0 TSval=39118
Frame 756: 1107 bytes on wire (8856 bits), 1107 bytes captured (8856 bits) on interface 0					
Ethernet II, Src: PcsCompu_09:a6:a4 (08:00:27:d9:a6:a4), Dst: PcsCompu_b3:3a:20 (08:00:27:b3:3a:20)					
Internet Protocol Version 4, Src: 192.168.56.102, Dst: 192.168.56.101					
Authentication Header					
Next header: TCP (6)					
Length: 6 (32 bytes)					
Reserved: 0000					
AH SPI: 0xc0d9eac9					
AH Sequence: 54					
AH ICV: f70bb5fe9f725d82d77af8f9db069eb4c0a83865					
Transmission Control Protocol, Src Port: 443, Dst Port: 207, Seq: 2833, Ack: 207, Len: 1009					
[3 Reassembled TCP Segments (2982 bytes): #752(1331), #754(1416), #756(235)]					
Secure Sockets Layer					
TLSv1.2 Record Layer: Handshake Protocol: Certificate					
Secure Sockets Layer					
TLSv1.2 Record Layer: Handshake Protocol: Server Key Exchange					
TLSv1.2 Record Layer: Handshake Protocol: Multiple Handshake Messages					

Figura 55: Bloque HTTPS protegido por AH

788	208.042893290..	192.168.56.101	192.168.56.102	TCP	106 48924 - 25 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1
789	208.043477591..	192.168.56.102	192.168.56.101	TCP	106 25 - 48924 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 S
790	208.043477591..	192.168.56.101	192.168.56.102	TCP	98 48924 - 25 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=396183 TS
791	208.045094171..	192.168.56.102	192.168.56.101	SMTP	167 S: 220 server ESMTP Exim 4.86.2 Ubuntu Sun, 26 Apr 2026 14:47:
792	208.045138492..	192.168.56.101	192.168.56.102	TCP	98 48924 - 25 [ACK] Seq=1 Ack=70 Win=29312 Len=0 TSval=396784 TS
793	208.071573868..	192.168.56.101	192.168.56.102	SMTP	121 C: EHLO [192.168.56.101]
794	208.072373197..	192.168.56.102	192.168.56.101	TCP	98 25 - 48924 [ACK] Seq=70 Ack=24 Win=29056 Len=0 TSval=396441 T
795	208.072607616..	192.168.56.102	192.168.56.101	SMTP	219 S: 250-server Hello [192.168.56.101] [192.168.56.101] [250-S
796	208.074363549..	192.168.56.101	192.168.56.102	SMTP	173 C: MAIL FROM:<daniel.93@fontanerosvillanueva6493.org> BODY=<B
797	208.075178180..	192.168.56.102	192.168.56.101	SMTP	106 S: 250 OK
798	208.112948604..	192.168.56.101	192.168.56.102	TCP	98 48924 - 25 [ACK] Seq=99 Ack=199 Win=29312 Len=0 TSval=396891
799	208.128759673..	192.168.56.101	192.168.56.102	SMTP	148 C: RCPT TO:<rafael.64@fontanerosvillanueva6493.org>
800	208.129606343..	192.168.56.102	192.168.56.101	SMTP	112 S: 250 Accepted
801	208.129649043..	192.168.56.101	192.168.56.102	TCP	98 48924 - 25 [ACK] Seq=149 Ack=213 Win=29312 Len=0 TSval=396808
802	208.129892181..	192.168.56.102	192.168.56.101	TCP	104 C: DATA
803	208.129982415..	192.168.56.102	192.168.56.101	SMTP	154 S: 354 Enter message, ending with "." on a line by itself
804	208.131058319..	192.168.56.101	192.168.56.102	SMTP	575 C: DATA fragment, 477 bytes
805	208.131244544..	192.168.56.101	192.168.56.102	SMTP[INF]	101 from: daniel.93 <daniel.93@fontanerosvillanueva6493.org>, sub
806	208.131757775..	192.168.56.102	192.168.56.101	TCP	98 25 - 48924 [ACK] Seq=209 Ack=635 Win=30800 Len=0 TSval=396458
807	208.140418574..	192.168.56.102	192.168.56.101	SMTP	126 S: 250 OK id=1w6ytx-0090xt-NP
808	208.141493920..	192.168.56.102	192.168.56.101	SMTP	104 C: QUIT
Frame 796: 173 bytes on wire (1384 bits), 173 bytes captured (1384 bits) on interface 0					
Ethernet II, Src: PcsCompu_b3:3a:20 (08:00:27:b3:3a:20), Dst: PcsCompu_09:a6:a4 (08:00:27:d9:a6:a4)					
Internet Protocol Version 4, Src: 192.168.56.101, Dst: 192.168.56.102					
Authentication Header					
Next header: TCP (6)					
Length: 6 (32 bytes)					
Reserved: 0000					
AH SPI: 0xc0dc4d98					
AH Sequence: 85					
AH ICV: 7be215290fe06a32368fa264e727464ec0a83866					
Transmission Control Protocol, Src Port: 48924, Dst Port: 25, Seq: 24, Ack: 191, Len: 75					
Simple Mail Transfer Protocol					

Figura 56: Bloque SMTP protegido por AH

816	211.784885349..	192.168.56.101	192.168.56.102	DNS	124 Standard query 0xb816 A pop.fontanerosvillanueva6493.org
817	211.784929261..	192.168.56.101	192.168.56.102	TCP	124 Standard query 0xf3ba AAAA pop.fontanerosvillanueva6493.org
818	211.785272302..	192.168.56.102	192.168.56.101	TCP	106 110 - 39982 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 S
819	211.785301335..	192.168.56.101	192.168.56.102	TCP	98 39982 - 110 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=397719 TS
820	211.785614852..	192.168.56.102	192.168.56.101	DNS	210 Standard query response 0xb816 A pop.fontanerosvillanueva6493.
821	211.785623781..	192.168.56.102	192.168.56.101	DNS	170 Standard query response 0xf3ba AAAA pop.fontanerosvillanueva6
822	211.791314048..	192.168.56.102	192.168.56.101	POP	118 S: +OK Dovecot ready.
823	211.791932195..	192.168.56.101	192.168.56.102	TCP	98 39982 - 110 [ACK] Seq=1 Ack=21 Win=29312 Len=0 TSval=397720 T
824	211.792261575..	192.168.56.101	192.168.56.102	POP	104 C: CAPA
825	211.792647782..	192.168.56.102	192.168.56.101	TCP	98 110 - 39982 [ACK] Seq=21 Ack=7 Win=29056 Len=0 TSval=397371 T
826	211.792663567..	192.168.56.102	192.168.56.101	POP	181 S: +OK
827	211.792987936..	192.168.56.101	192.168.56.102	POP	119 C: AUTH PLAIN
828	211.793522319..	192.168.56.102	192.168.56.101	POP[INF]	102
829	211.793584452..	192.168.56.101	192.168.56.102	POP	120 C: AHJhZmf1bF8ZNAAXMJM0
830	211.800826890..	192.168.56.102	192.168.56.101	POP	114 S: +OK Logged in.
831	211.800902022..	192.168.56.101	192.168.56.102	POP	104 C: STAT
832	211.805968038..	192.168.56.102	192.168.56.101	POP	119 S: +OK 7 5991
833	211.806456784..	192.168.56.101	192.168.56.102	POP	104 C: LIST
834	211.806692808..	192.168.56.102	192.168.56.101	POP	167 S: +OK 7 messages:
835	211.806705998..	192.168.56.101	192.168.56.102	POP	104 C: UIDL
836	211.806965547..	192.168.56.102	192.168.56.101	POP	246 S: +OK
837	211.807233954..	192.168.56.101	192.168.56.102	POP	106 C: RETR 7
838	211.807483720..	192.168.56.102	192.168.56.101	POP	979 S: +OK 862 octets
Frame 822: 118 bytes on wire (944 bits), 118 bytes captured (944 bits) on interface 0					
Ethernet II, Src: PcsCompu_09:a6:a4 (08:00:27:d9:a6:a4), Dst: PcsCompu_b3:3a:20 (08:00:27:b3:3a:20)					
Internet Protocol Version 4, Src: 192.168.56.102, Dst: 192.168.56.101					
Authentication Header					
Next header: TCP (6)					
Length: 6 (32 bytes)					
Reserved: 0000					
AH SPI: 0xc0d9eac9					
AH Sequence: 86					
AH ICV: c8513ba0a49a1ca2b3d744a0568de4c0a83965					
Transmission Control Protocol, Src Port: 110, Dst Port: 30089, Seq: 1, Ack: 1, Len: 70					

Figura 57: Bloque POP protegido por AH

Cuando se realiza un ping desde al cliente a la máquina servidor, al seleccionar un paquete de ida y de vuelta se puede observar el SPI de cada uno. El SPI identifica la asociación de seguridad usada en cada sentido y, como las asociaciones de IPsec son unidireccionales, el tráfico de cliente-servidor y servidor-cliente tienen SPI distintos, como se puede observar en las siguientes imágenes.

1263	1117457.9212068	192.168.56.101	192.168.56.102	ICMP	130 Echo (ping) request	id=0x0a77, seq=1/256 (reply in 1264)
1264	1117457.9219748	192.168.56.102	192.168.56.101	ICMP	130 Echo (ping) reply	id=0x0a77, seq=1/256 (request in 1263)

```

▶ Frame 1263: 130 bytes on wire (1040 bits), 130 bytes captured (1040 bits) on interface 0
▶ Ethernet II, Src: PcsCompu_b3:3a:20 (08:00:27:b3:3a:20), Dst: PcsCompu_d9:a6:a4 (08:00:27:d9:a6:a4)
▶ Internet Protocol Version 4, Src: 192.168.56.101, Dst: 192.168.56.102
▼ Authentication Header
  Next header: ICMP (1)
  Length: 6 (32 bytes)
  Reserved: 0000
  AH SPI: 0xc6fddedab
  AH Sequence: 204
  AH ICV: 9b4527a765e387ab6ba7561a5b47cc10c9a83866

```

Figura 58: Ping de cliente a servidor protegido por AH y visualización del SPI

1263	1117457.9212068	192.168.56.101	192.168.56.102	ICMP	130 Echo (ping) request	id=0x0a77, seq=1/256 (reply in 1264)
1264	1117457.9219748	192.168.56.102	192.168.56.101	ICMP	130 Echo (ping) reply	id=0x0a77, seq=1/256 (request in 1263)

```

▶ Frame 1264: 130 bytes on wire (1040 bits), 130 bytes captured (1040 bits) on interface 0
▶ Ethernet II, Src: PcsCompu_d9:a6:a4 (08:00:27:d9:a6:a4), Dst: PcsCompu_b3:3a:20 (08:00:27:b3:3a:20)
▶ Internet Protocol Version 4, Src: 192.168.56.102, Dst: 192.168.56.101
▼ Authentication Header
  Next header: ICMP (1)
  Length: 6 (32 bytes)
  Reserved: 0000
  AH SPI: 0xc95f8dca
  AH Sequence: 199

```

Figura 59: Respuesta al ping y visualización del SPI

1269	1117458.9225328	192.168.56.101	192.168.56.102	ICMP	130 Echo (ping) request	id=0x0a77, seq=2/512 (reply in 1270)
1270	1117458.9233215	192.168.56.102	192.168.56.101	ICMP	130 Echo (ping) reply	id=0x0a77, seq=2/512 (request in 1269)

```

▶ Frame 1269: 130 bytes on wire (1040 bits), 130 bytes captured (1040 bits) on interface 0
▶ Ethernet II, Src: PcsCompu_b3:3a:20 (08:00:27:b3:3a:20), Dst: PcsCompu_d9:a6:a4 (08:00:27:d9:a6:a4)
▶ Internet Protocol Version 4, Src: 192.168.56.101, Dst: 192.168.56.102
▼ Authentication Header
  Next header: ICMP (1)
  Length: 6 (32 bytes)
  Reserved: 0000
  AH SPI: 0xc6fddedab
  AH Sequence: 207
  AH ICV: 6fab9db6a5ddade75db46bfe68f5d45c0a83866

```

Figura 60: Otro ping con el mismo SPI

1269	1117458.9225328	192.168.56.101	192.168.56.102	ICMP	130 Echo (ping) request	id=0x0a77, seq=2/512 (reply in 1270)
1270	1117458.9233215	192.168.56.102	192.168.56.101	ICMP	130 Echo (ping) reply	id=0x0a77, seq=2/512 (request in 1269)

```

▶ Frame 1270: 130 bytes on wire (1040 bits), 130 bytes captured (1040 bits) on interface 0
▶ Ethernet II, Src: PcsCompu_d9:a6:a4 (08:00:27:d9:a6:a4), Dst: PcsCompu_b3:3a:20 (08:00:27:b3:3a:20)
▶ Internet Protocol Version 4, Src: 192.168.56.102, Dst: 192.168.56.101
▼ Authentication Header
  Next header: ICMP (1)
  Length: 6 (32 bytes)
  Reserved: 0000
  AH SPI: 0xc95f8dca
  AH Sequence: 202
  AH ICV: e4de4d805f914d0dec08688d5759cbd0c0a83865

```

Figura 61: Respuesta del servidor con el mismo SPI de antes

9. Problemas encontrados

En el servidor HTTP desarrollado en Python, hay un problema con la gestión de cookies, ya que el contador de accesos se incrementa por cada recurso que se solicita al servidor. Por ejemplo, al acceder a la página principal, el navegador también solicita la imagen que está incluida en la página, haciendo que el contador aumente más de una vez. Esto se debe a que el navegador realiza una petición HTTP para cada recurso. Aunque esto permite observar el funcionamiento de la cookie y el bloqueo mediante **403 Forbidden**, sigue siendo una limitación, ya que lo correcto sería que el contador se actualice únicamente cuando se solicita el recurso principal.

También, hemos tenido que quitar bastante contenido de la memoria ya que nos pasamos del límite de 40 páginas, por lo que hemos tenido que dedicar más tiempo en la realización de este documento para poder adaptarlo al límite de páginas.

10. Horas empleadas (aproximadas)

Apartado	Horas aproximadas
Programación Web-SSTT HTTP Server	17
DNS	5
SMTP/POP	4
HTTP/HTTPS	10
IPsec	6
Documentación	12
Total	54

11. Conclusión y valoración personal

Siendo totalmente sinceros, la práctica nos ha parecido bastante interesante, ya que es una de las pocas que hemos realizado que realmente se acercan a un proyecto que podríamos encontrarnos en un entorno laboral real. No ha sido simplemente configurar o programar un código, sino montar un escenario completo y con varios servicios que funcionan de manera conjunta y coherente.

Además, una de las cosas que más nos ha gustado es que nos ha permitido ver lo que hay realmente detrás de algo tan normal como el acceder a una página web o usar un servicio de correo. El tener que configurar los protocolos y hacer que funcionen conjuntamente ha hecho que podamos entender mejor el papel que tiene cada uno.

En general, esta práctica no nos ha resultado muy complicada ya que los boletines de prácticas nos guiaban bastante bien en el desarrollo y explicaban de una manera bastante fácil de entender y seguir lo que había que hacer. Aun así, eso no quita que hayamos tenido que leer documentación sobre los distintos protocolos para poder entender mejor qué estábamos haciendo y por qué para poder entenderlo y documentarlo bien.

También, el desarrollo de esta práctica nos ha venido bien como apoyo para entender mejor el contenido de la parte de teoría de la asignatura ya que estos temas están directamente relacionados con todo el desarrollo de la práctica, de forma que, lo que se imparte en las clases de teoría teníamos que aplicarlo para el desarrollo de la práctica.

En resumen, nos ha parecido una práctica bastante completa, útil e interesante, porque nos ha enseñado a montar un escenario funcional con varios servicios y a entender mejor lo que ocurre por detrás.